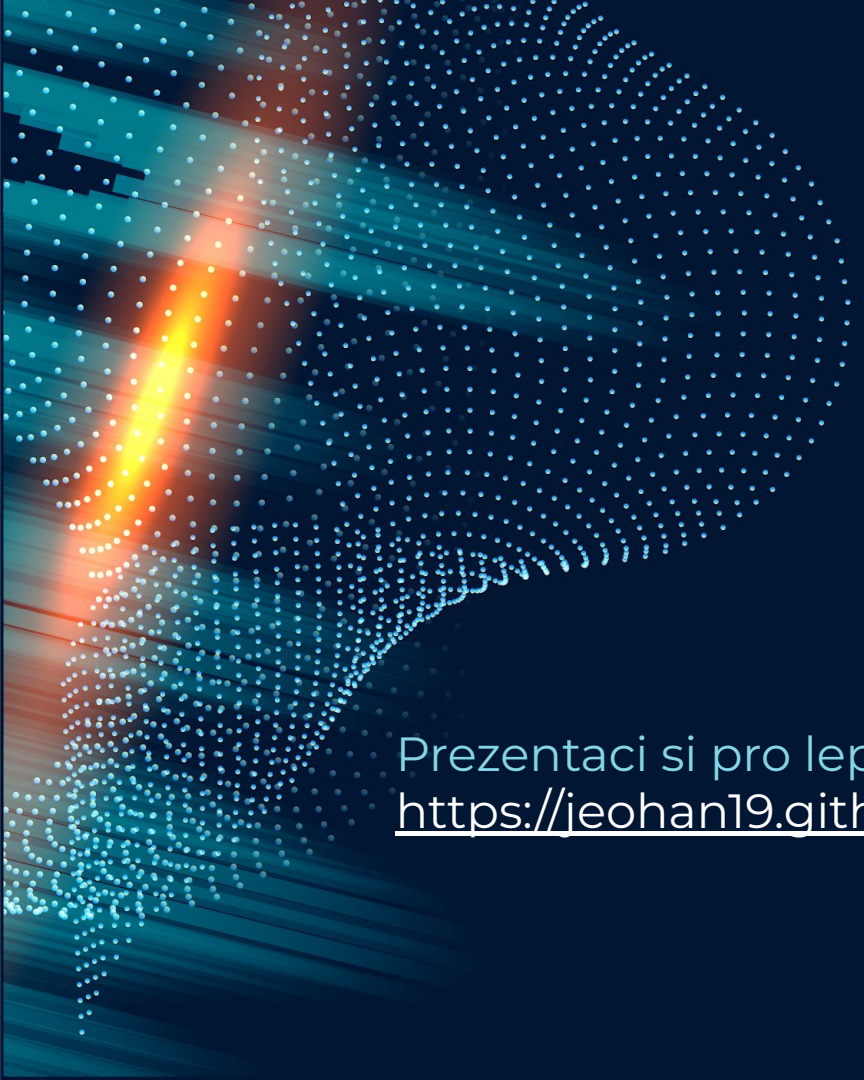


# Modely pro predikci nelineárních vztahů

Johan Eigner R3.A



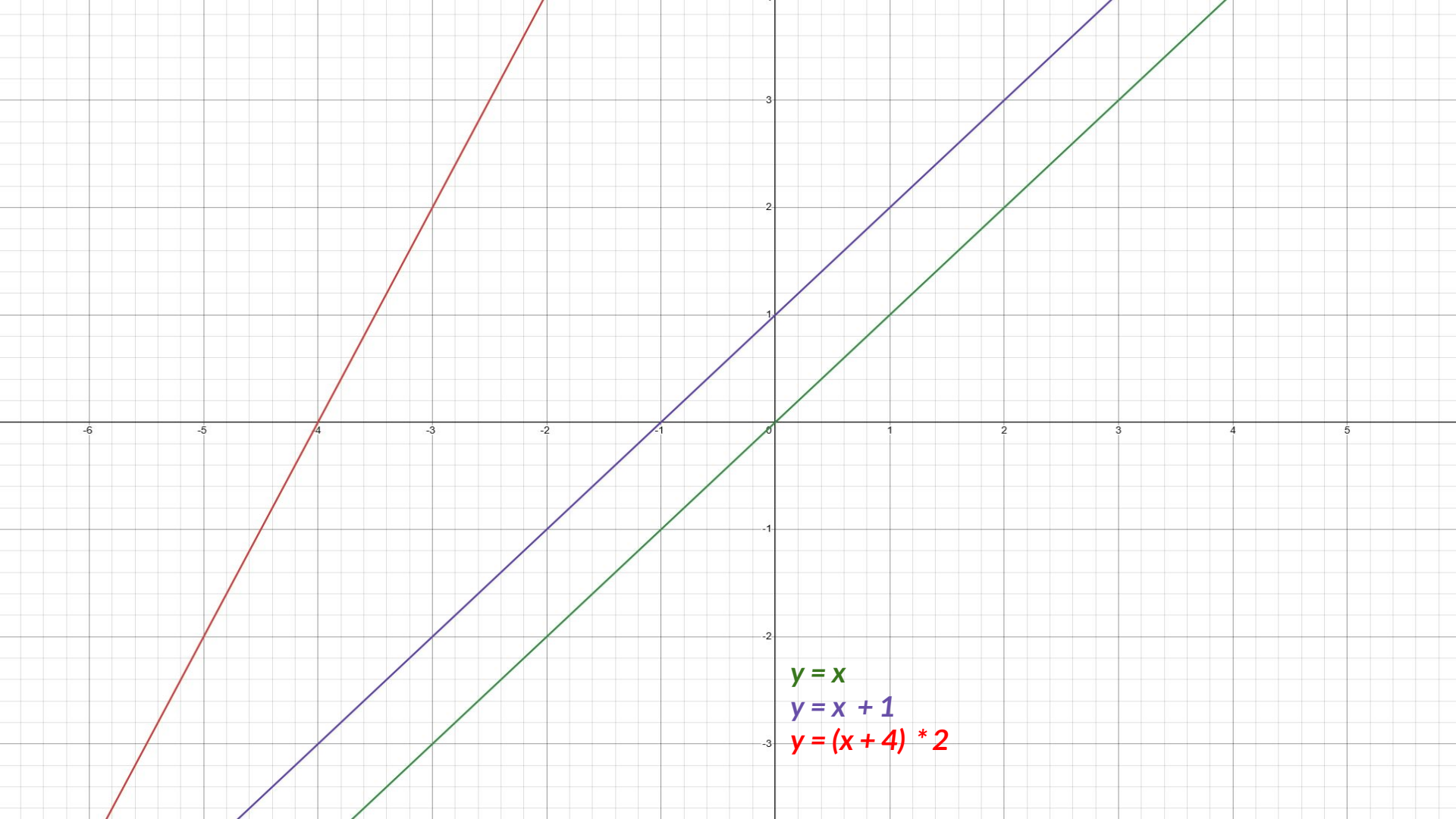
# Úvod a teorie

Prezentaci si pro lepší čitelnost můžete spustit na  
[https://jeohan19.github.io/inf\\_MLP\\_prezentace/](https://jeohan19.github.io/inf_MLP_prezentace/)

# Lineární vztah

- y se mění v závislosti na x
- grafem bude vždy přímka
- například  $y = x * 2$  nebo  $y = (x + 3) / 7$
- každý lineární vztah lze vyjádřit jako  $y = a * x + b$
- pokud máme sadu x, y jsme schopni vztah pomocí rovnice odvodit
- $a$  určuje sklon přímky,  $b$  určuje posun





$$y = x$$

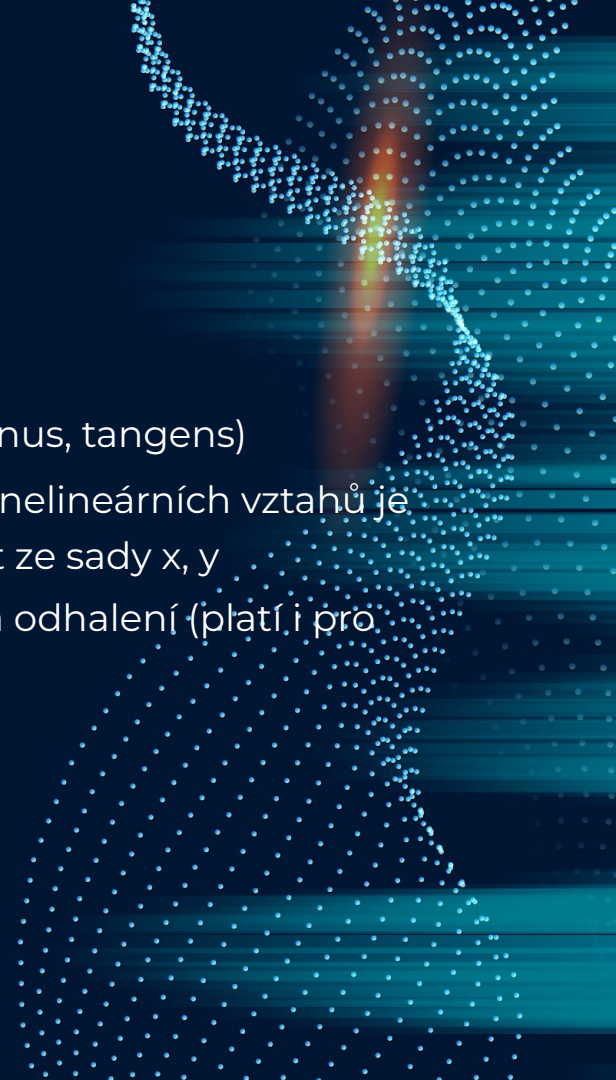
$$y = x + 1$$

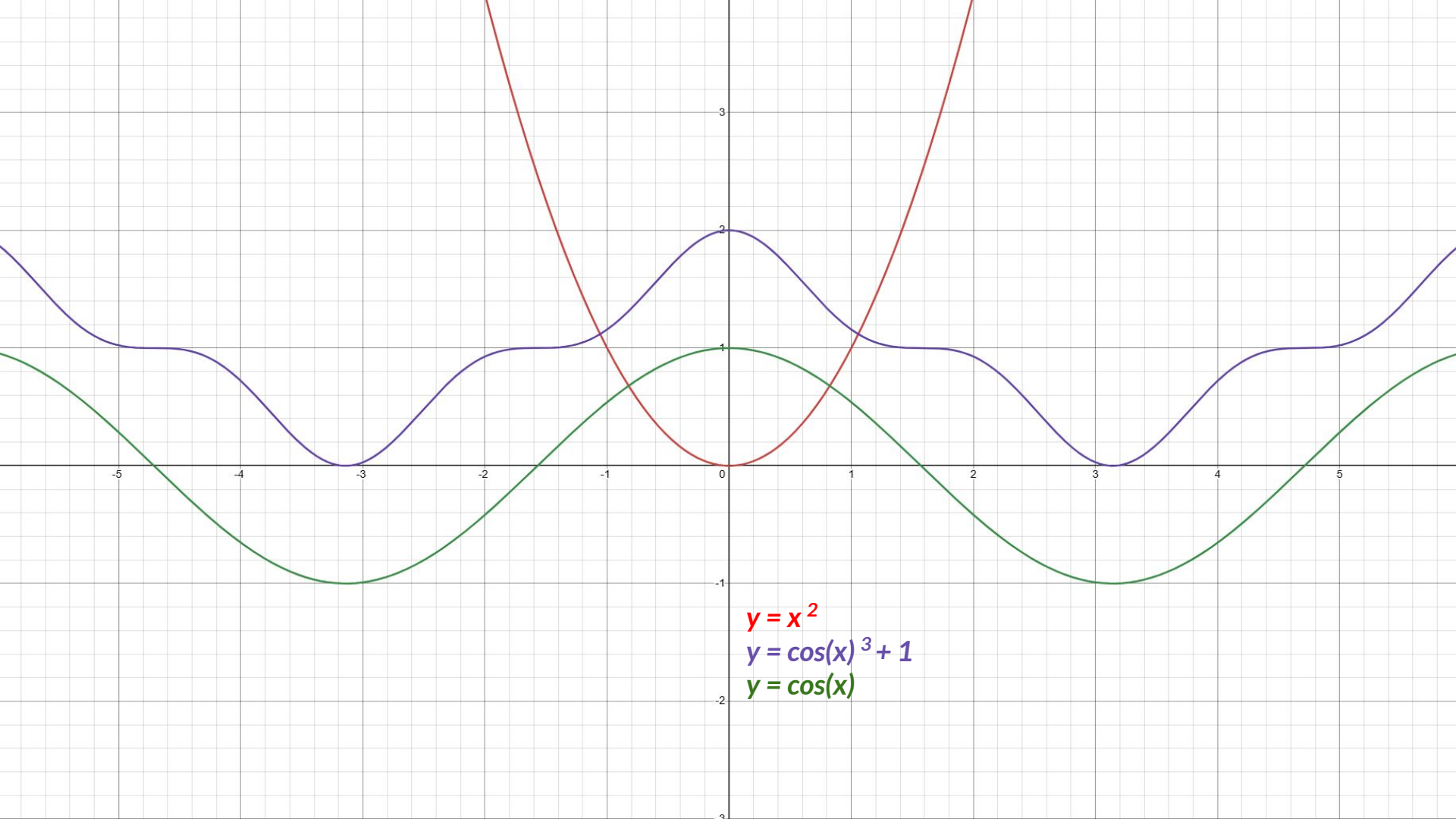
$$y = (x + 4) * 2$$



# Nelineární vztah

- grafem není přímka
- nelze vyjádřit jako  $y = a * x + b$
- například mocniny, goniometrické funkce (sinus, cosinus, tangens)
- u složitějšího nelineárního vztahu nebo u kombinace nelineárních vztahů je obtížné nebo nemožné je nějak matematicky odvodit ze sady  $x, y$
- čím “křivější” graf daný vztah má, tím náročnější je na odhalení (platí i pro neuronové sítě)

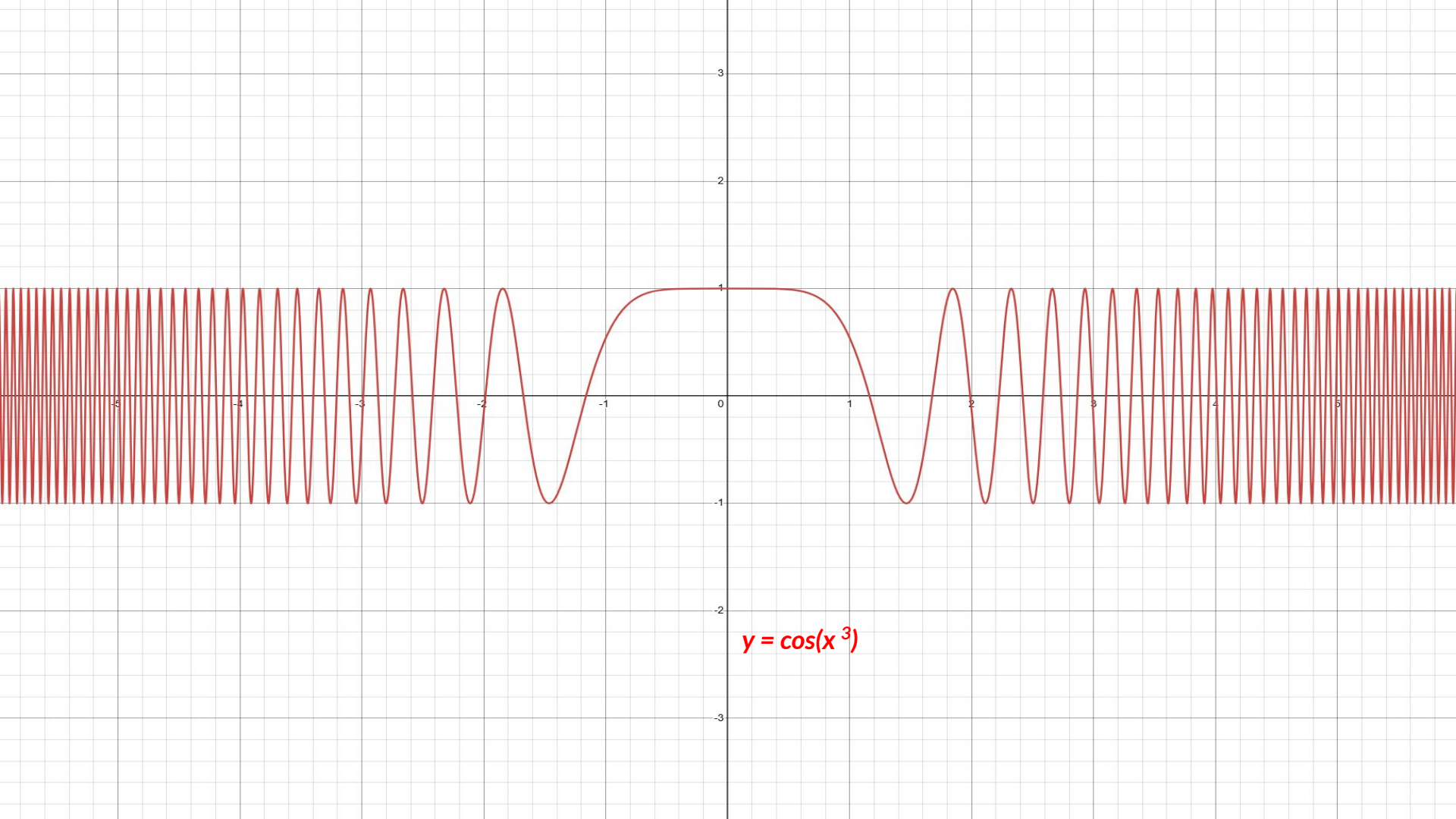




$$y = x^2$$

$$y = \cos(x)^3 + 1$$

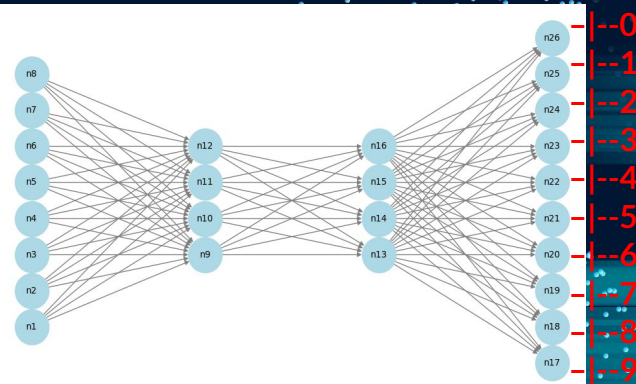
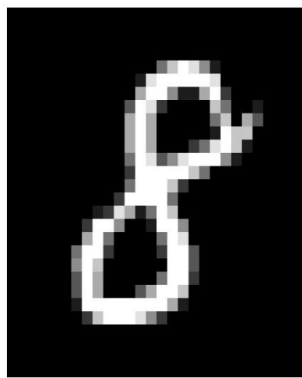
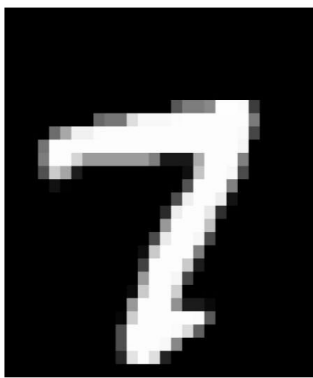
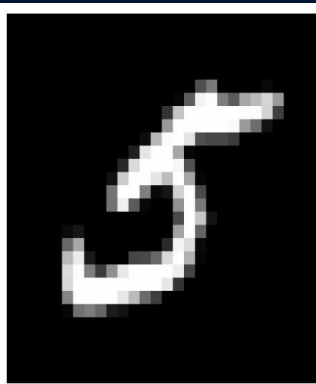
$$y = \cos(x)$$

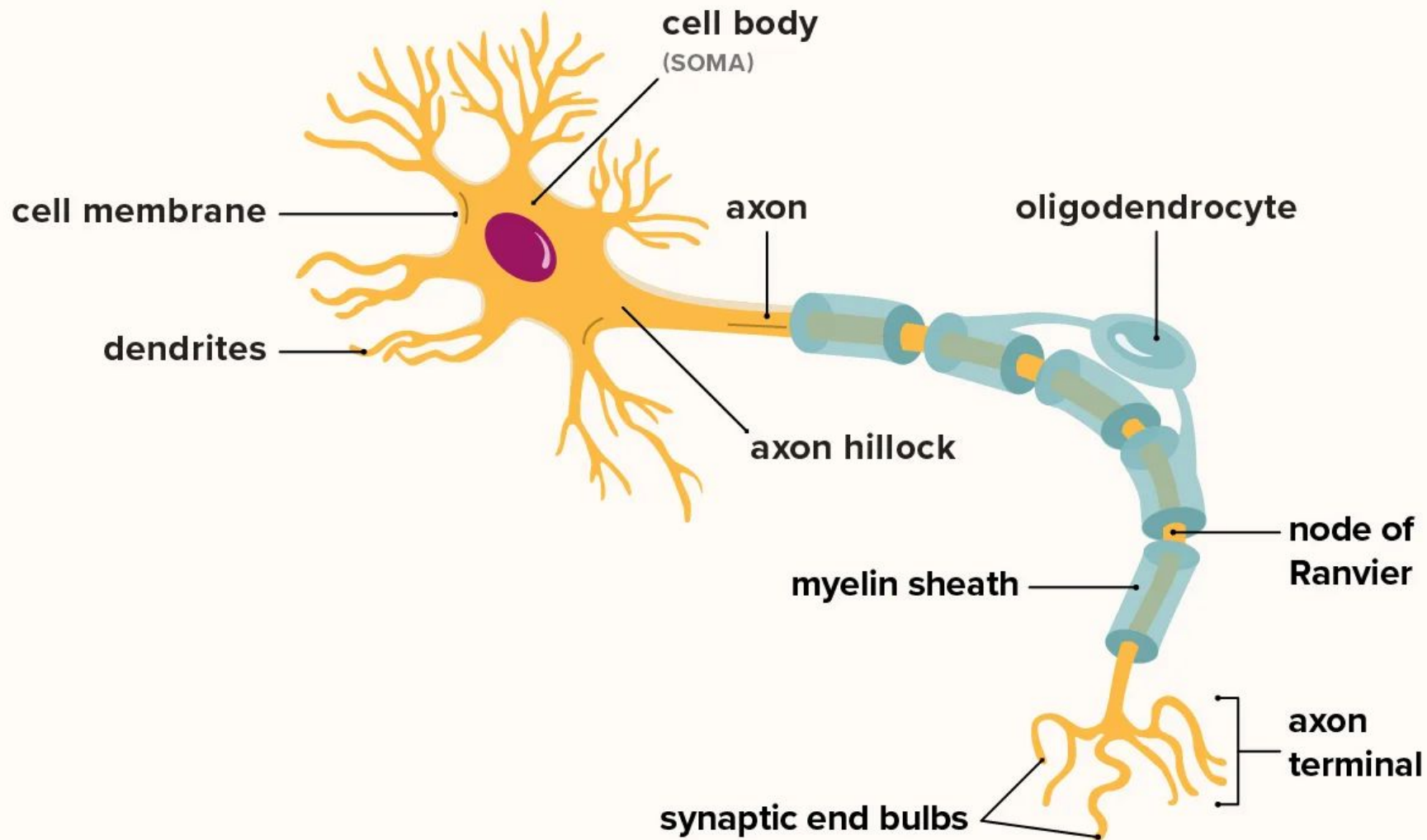


$$y = \cos(x^3)$$

# Nelineární úlohy

- běžnými algoritmy je řešení extrémně náročné / nemožné
- řešení je nelineární nebo není založeno na matematických principech
- řeší se pomocí různých typů neuronových sítí
- odvození znaků z rukopisu → systém psaných číslic nevznikl na principech matematiky
- zpracování přirozeného jazyka







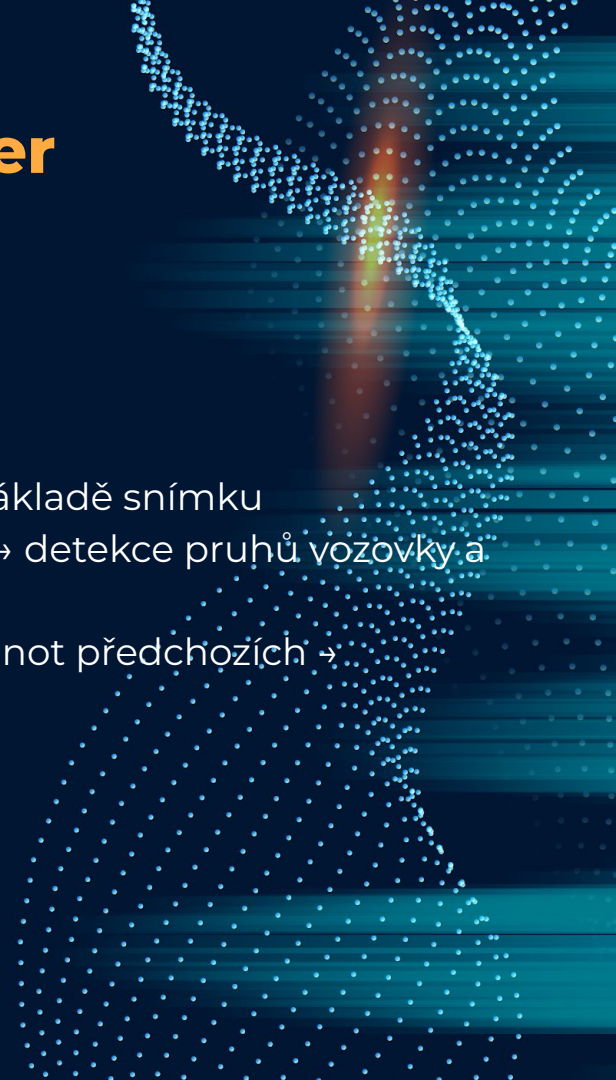


# Architektura neuronové sítě

- původně založena na principech biologických neuronových sítí
- nyní se v některých případech původnímu konceptu vzdaluje → stále založena na neuronech, ale čím dál více je koncept modifikován pro nižší výpočetní náročnost a lepší uzpůsobení dané úloze
- několik základních architektur pro různé úlohy:
  - vícevrstvý perceptron → až v další části prezentace
  - konvoluční neuronové sítě → zpracování obrazu
  - rekurentní neuronové sítě → zpracování sekvenčních dat (data s časovou závislostí) - Google Translate, rozpoznání řeči, předpověď akciového trhu
    - využívá spoje mezi vzdálenými vrstvami
  - transformerové sítě → nejpokročilejší, Chat GPT (první ze všech), Microsoft Copilot a další

# Vícevrstvý perceptron (MultiLayer Perceptron)

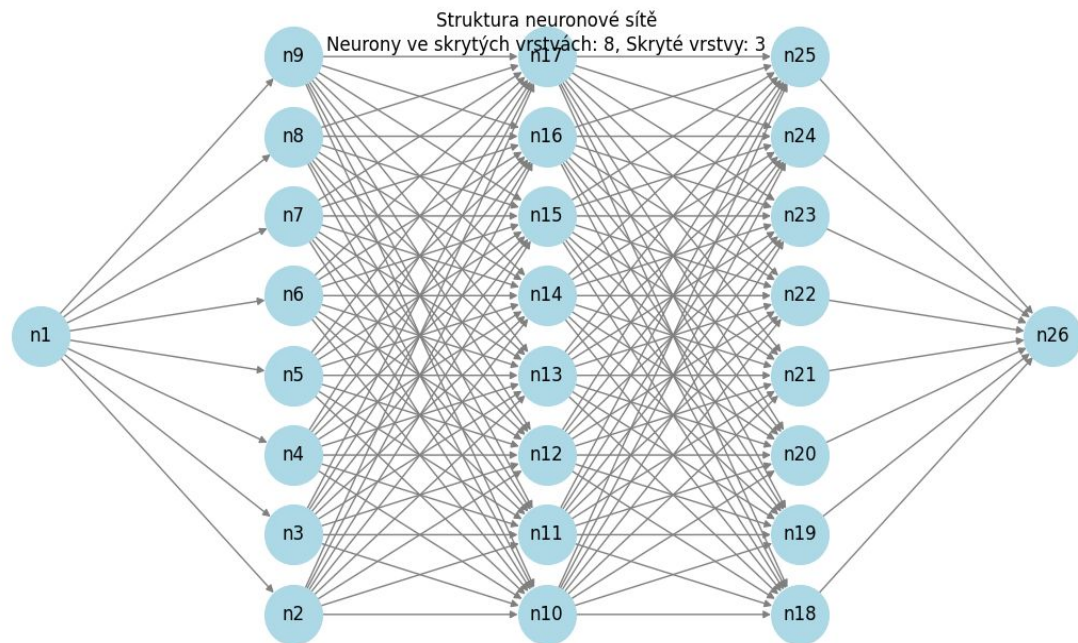
- základ neuronových sítí a ai
- první neuronová síť schopná zachytit nelineární vztah
- princip znám již v 70. letech
- rozpoznání rukopisného textu
- automatizovaná výroba → detekce vadných kusů na základě snímku
- jednodušší autonomní systémy → autopiloti v autech → detekce pruhů vozovky a udržování vozidla mezi nimi
- regrese (predikce následující hodnoty na základě hodnot předchozích → například akciový trh)





# Architektura a prvky vícevrstvého perceptronu

- vstupní vrstva
- skryté vrstvy
- výstupní vrstva
- aktivační funkce
- neurony
- váhy
- biasy
- inicializace vah a biasů
- algoritmy učení



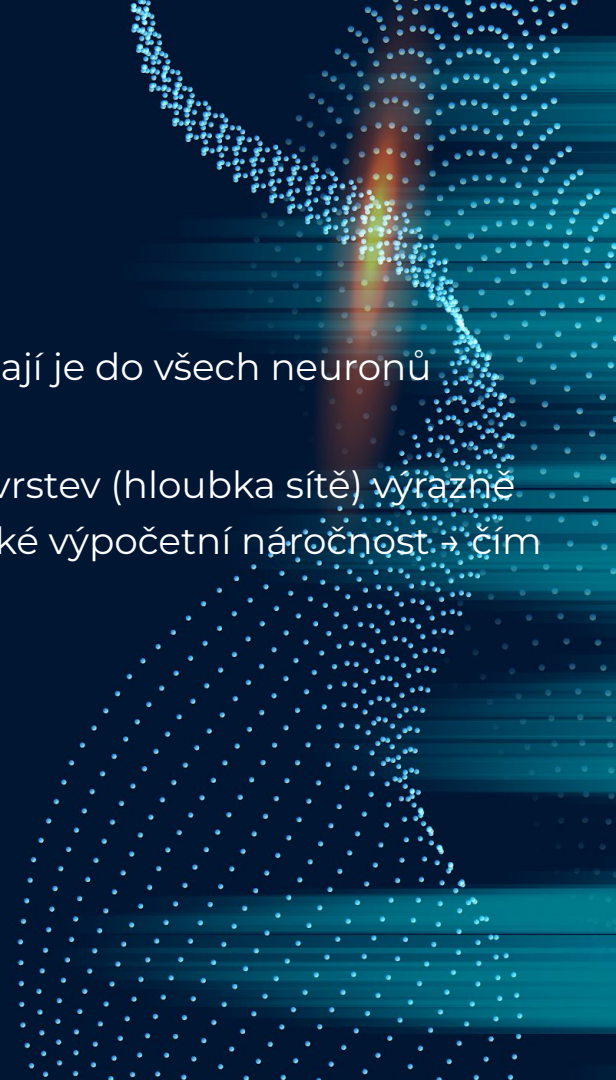
# Vstupní vrstva

- každý neuron ve vstupní vrstvě přijímá jednu hodnotu → počet informací se kterými síť pracuje = počet neuronů ve vstupní vrstvě
- pokud síť pracuje s ku příkladu fotografií, každý neuron ve vstupní vrstvě přijme hodnotu jednoho pixelu z fotografie
  - pokud síť zpracovává obrázek  $28 * 28$  pixelů ve vstupní vrstvě bude  $28 * 28 = 784$  neuronů
- pokud pracuje s regresí (predikce  $y$  v závislosti na  $x$ ) → jeden vstupní neuron (naš případ)
- každý neuron ve vstupní vrstvě je propojen s každým neuronem ve vrstvě následující



# Skryté vrstvy

- vrstvy uvnitř sítě
- přijímají vstupy, provádí s nimi danou operaci a odesílají je do všech neuronů následující vrstvy
- počet neuronů ve skrytých vrstvách a počet skrytých vrstev (hloubka sítě) výrazně ovlivňují složitost vztahu, který se síť může naučit a také výpočetní náročnost → čím více, tím náročnější na výkon



# Výstupní vrstva

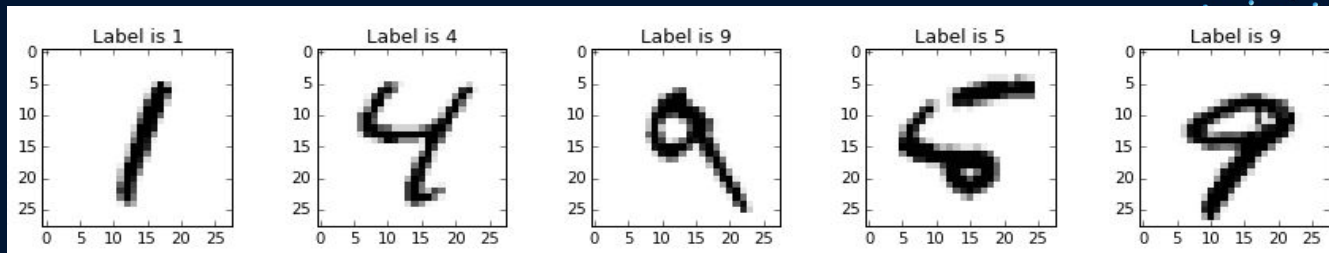
- počet neuronů závisí na úloze
- pokud je úlohou regrese → výstup bude jedna číselná hodnota → 1 neuron
- pokud má neuronová síť vybrat z více možností (například která číslice je na fotce), počet neuronů bude roven počtu možných odpovědí
  - výsledek každého neuronu bude pravděpodobnost, že číslo na fotce je číslice, které neuron reprezentuje
  - pokud není v síti chyba, nejvyšší pravděpodobnost by měl dát neuron, který reprezentuje číslici na fotce

# Aktivační funkce

- stanovuje tzv. aktivační práh
- pokud je hodnota vstup do neuronu pod touto hranicí, neuron se vypne a dál žádný signál nepošle
- průlomový koncept
- přináší nepravidelnost (nelinearitu) → pokud se některé neurony zdánlivě náhodně vypnou, síť umí pracovat i s nepravidelnými vzory, které neodpovídají matematickým principům
- díky aktivační funkci (zjednodušeně, svoji roli hrají i některé další faktory) jsou schopné neuronové sítě svých jedinečných výkonů

# Aktivační funkce - příklad

- pokud síť dostane černobílý obrázek číslice, bílé pozadí nevzbudí v síti žádnou odezvu → neurony se neaktivují
- aktivační funkce umožní síti reagovat pouze na obrys číslice
- díky tomu je síť schopná “vidět / soustředit se na” důležité informace a zbytek ignorovat





# ReLU a Leaky ReLU

**ReLU** < *pokud je výstup neuronu < 0:  $\text{ReLU}(n) = 0$  → problém mrtvých neuronů*  
- *pokud je výstup neuronu > 0:  $\text{ReLU}(n) = n$*

**LReLU** < *pokud je výstup neuronu < 0:  $\text{ReLU}(n) = n * \text{alfa}$  (například 0.01)*  
- *pokud je výstup neuronu > 0:  $\text{ReLU}(n) = n$*

- řeší problém mrtvých neuronů → sníží výstupní hodnotu ale neuron se nepřestane učit

# Váhy

- hodnota každého spojení mezi neurony
- založeno na principu synaps v biologických neuronových sítích
- značeno jako  **$w$**



# Biasy

- posunují aktivační funkce neuronů
- umožňují neuronu aktivovat se i při nižších
- lepší přizpůsobení sítě datům (uvidíme v následujících částech prezentace)
- značeny jako  **$b$**



# Mraveniště aneb co všechno musí každý neuron udělat...

- zbytek prezentace se vztahuje pouze na menší regresní modely (jeden vstupní neuron, skryté vrstvy, jeden výstupní) → něco víc by bylo na další (ještě obsáhlejší) prezentaci
- každý neuron v každé vrstvě provádí přibližně toto:
  - **výstup neuronu = aktivací funkce ( $w_1 * n_1 + w_2 * n_2 + \dots + w_n * n_n + b$ )**
  - $b$  = bias
  - $w$  = váha spoje
  - $n$  = hodnota, kterou odeslaly neurony v předchozí vrstvě
  - výstup neuronu je poté odeslán všem neuronům v další vrstvě (pokud projde aktivací funkcí)



# Inicializace vah

- každý neuron odešle informaci do všech neuronů v následující vrstvě
- čísla porostou extrémně rychle → každý neuron v následující vrstvě obsahuje všechny hodnoty z předchozí vrstvy → následující vrstva bude mít hodnoty v podstatě o rozměrech **hodnoty předchozí vrstvy \* počet neuronů v následující vrstvě**
- čísla explodují
- je nutné počáteční hodnoty nastavit s ohledem na tento jev, nikoliv náhodně
  - ale jak?
  - (Kaimingova inicializace)

# Inicializace vah a biasů

- $n_1$  = rozsah -1:1 (průměr 0)
- $w$  = náhodný rozsah -1:1 (průměr 0)
- pro jednoduchost všechny váhy, vstupy jsou stejné
- kvadratická chyba  $\rightarrow$  všechny hodnoty kladné
- $n$  = součet energie neuronů se vstupem do daného neuronu
- $E[\text{neuron}] = E[(n * w)^2] = w^2 * E[n^2]$
- $E[\text{vrstva s } N \text{ neurony}] = N * w^2 * E[n^2]$
- $E[\text{předchozí vrstva}] = E[\text{následující vrstva}]$
- $N * w^2 * 1 = 1$ 
  - předpokládáme:  $E[\text{předchozí vrstva}]$  je 1
- $w^2 = 1 / N \rightarrow w = \sqrt{1 / N}$ 
  - odmocnění obou stran rovnic



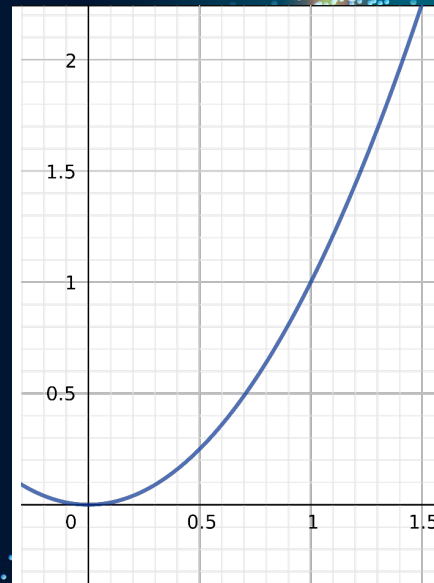
# Inicializace vah

- závěr:
  - každá vrstva bude mít výstupní energii podobnou jako ta předchozí
  - nevznikají nekonečné hodnoty, které způsobují zhroucení sítě
  - ke správnému nastavení musíme znát počet neuronů předchozí vrstvy a počet neuronů následující vrstvy
  - polovinu neuronů “zabije” ReLU, tedy:  $w = \sqrt{1 / N} \rightarrow$

$$w = \sqrt{\frac{2}{N}}$$

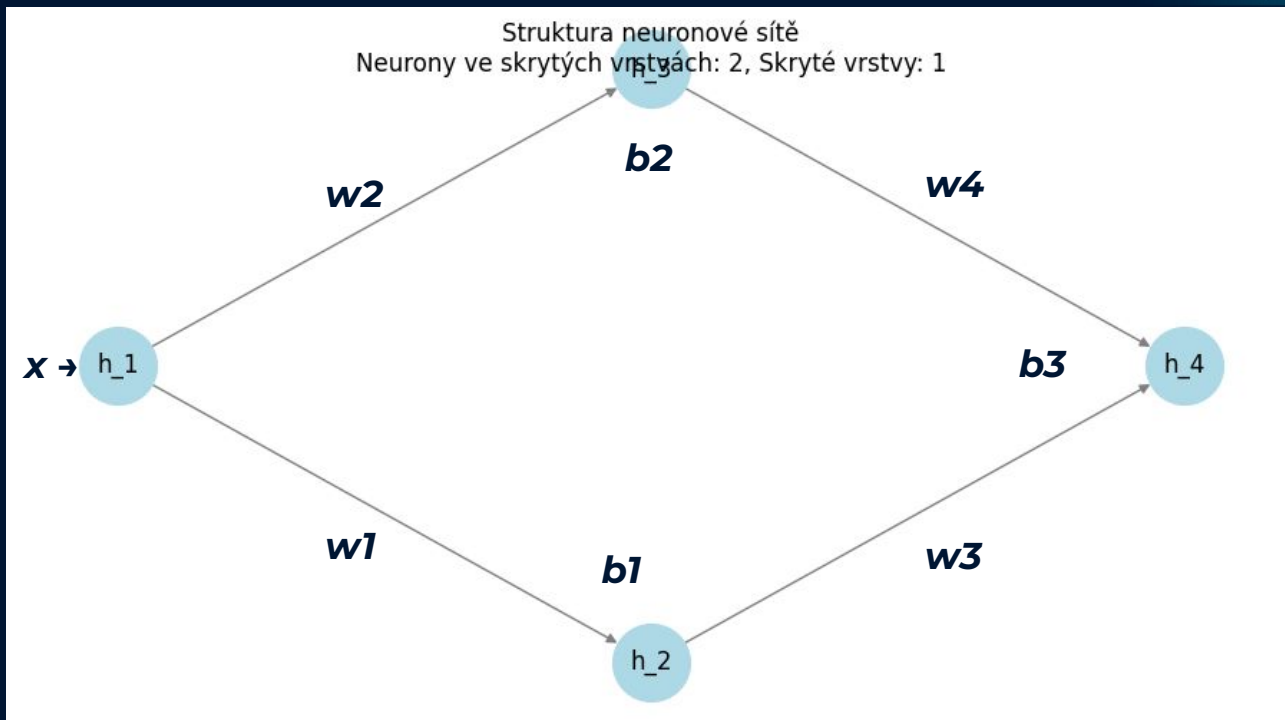
# Algoritmy učení (zpětné šíření chyby)

- díky učícím algoritmům je síť upravována z počátečních náhodných vah pro lepší výsledky
- zpětné šíření chyby nebo také zpětná propagace chyby  
→ jeden z nejstarších a nejpoužívanějších systémů
- kvadratická chyba ( $\text{Loss}^2$ )
  - eliminuje záporné hodnoty
  - čím větší bude odchylka, tím více se umocní
- mini-batch
  - neaktualizuje parametry sítě po každém příkladu, ale aktualizuje je vždy o průměr z třeba 16 příkladů (kvůli menší náročnosti na výkon)
- následující podrobnosti jsou uvedeny čistě pro úplnost



# V praxi

- pro ilustraci si vezmeme tuto síť:





# Derivace (velmi zjednodušeně)

- udává rychlost změny výsledku funkce vůči změně konkrétní proměnné
- příklad: **derivace** ( $w * n + b$ ) **podle**  $w = n \rightarrow d/dw(w * n + b) = n$  **nebo**  $d/dw = n$ 
  - hodnota derivace \* změna proměnné = změna výsledku funkce
  - $w = 2, n = 2, b = 2 \rightarrow 2 * 2 + 2 = 6$
  - $w = 3, n = 2, b = 2 \rightarrow 3 * 2 + 2 = 8$
  - **změna  $w = 1$**
  - $n * 1 = 2 = 8 - 6$

# Algoritmy učení (zpětné šíření chyby)

- využívá řetězovou derivaci k zjištění, jak změna dané hodnoty ovlivní výsledek sítě
- podle toho hodnoty upravuje
- **$x$  = vstupní hodnota**
- **$b1, b2$ : bias pro první a druhý skrytý neuron**
- **$b3$ : bias pro výstupní neuron**
- **$t$ : cílová hodnota (target)**
- **$w1$ : váha od vstupu k prvnímu skrytému neuronu**
- **$w2$ : váha od vstupu ke druhému skrytému neuronu**
- **$w3$ : váha od prvního skrytého neuronu k výstupu**
- **$w4$ : váha od druhého skrytého neuronu k výstupu**

# Algoritmy učení (zpětné šíření chyby)

- $L = \text{kvadratická chyba} = \frac{1}{2} * (y - t)^2$ 
  - *derivace mocniny*  $= 2x$
  - *derivace*  $L = \frac{1}{2} * 2(y - t) = (y - t)$
  - **používá se pouze kvůli zjednodušení výpočtu**
- $\text{ReLU}(z) = \max(0, z)$   $d\text{ReLU}(z)/dz = 1$ , pokud  $z > 0$ 
  - $= 0$ , pokud  $z \leq 0$
- $z1 = w1 * x + b1$ : předaktivace prvního skrytého neuronu
- $h1 = \text{ReLU}(z1)$ : výstup prvního skrytého neuronu
- $z2 = w2 * x + b2$ : předaktivace druhého skrytého neuronu
- $h2 = \text{ReLU}(z2)$ : výstup druhého skrytého neuronu
- $z3 = w3 * h1 + w4 * h2 + b3$ : předaktivace výstupního neuronu
- $y = z3$ : výstupní hodnota (bez aktivací funkce ve výstupu, lineární)
- $L = 0.5 * (y - t)^2$

# Řetězové derivace

- obecný tvar aktualizace vah a biasů:
  - $w_n = w_n - \text{learning\_rate} * dL/dw_n$
  - $b_n = b_n - \text{learning\_rate} * dL/db_n$
- 
- $dL/dw1 = dL/dy * dy/dh1 * dh1/dz1 * dz1/dw1$ 
  - $dL/dw1 = (y - t) * w3 * \text{ReLU}'(z1) * x$
- 
- $dL/dw2 = dL/dy * dy/dh2 * dh2/dz2 * dz2/dw2$ 
  - $dL/dw2 = (y - t) * w4 * \text{ReLU}'(z2) * x$
- 
- $dL/dw3 = dL/dy * dy/dw3$ 
  - $dL/dw3 = (y - t) * h1$
- 
- $dL/dw4 = dL/dy * dy/dw4$ 
  - $dL/dw4 = (y - t) * h2$

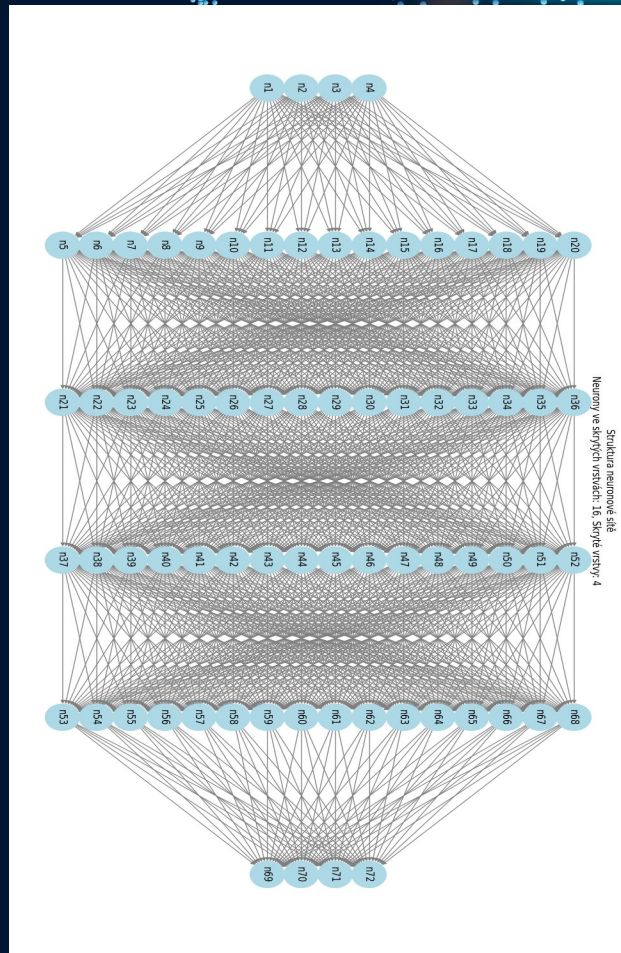
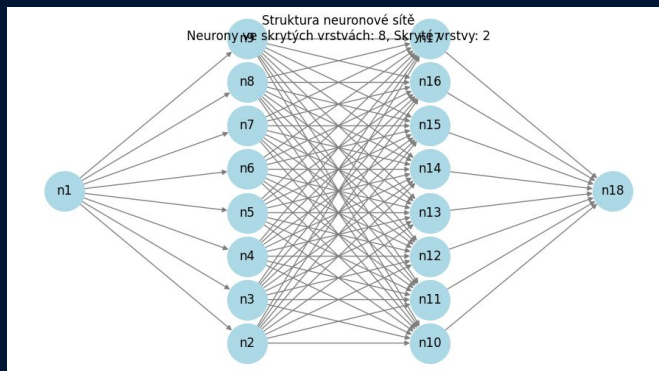
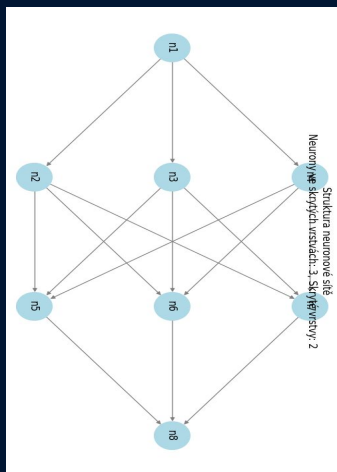
# Řetězové derivace

- $dL/db1 = dL/dy * dy/dh1 * dh1/dz1 * dz1/db1$ 
  - $dL/db1 = (y - t) * w3 * \text{ReLU}'(z1) * 1$
- 
- $dL/db2 = dL/dy * dy/dh2 * dh2/dz2 * dz2/db2$ 
  - $dL/db2 = (y - t) * w4 * \text{ReLU}'(z2) * 1$
- 
- $dL/db3 = dL/dy * dy/db3$ 
  - $dL/db3 = (y - t) * 1 = y - t$



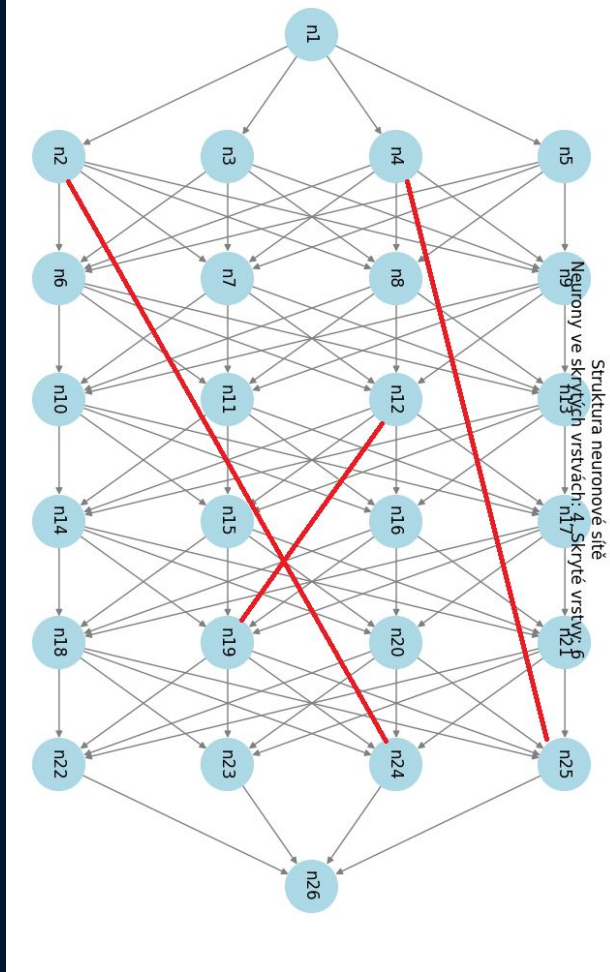
**Co se tedy dá zlepšit, když se princip od 70. let nezměnil aneb kam pokračuje vývoj (pokud se něco šeredně nepokazí...)**

- CHYTŘEJŠÍ...
- RYCHLEJŠÍ...
- ROZSÁHLEJŠÍ...



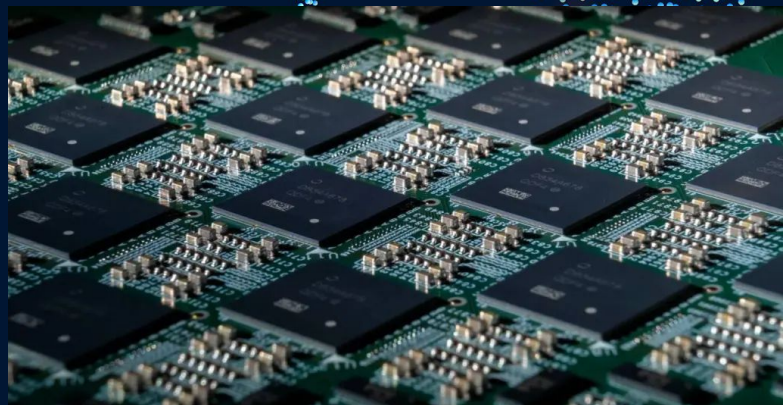
# Chytřejší...

- princip zůstává, ale stále se dá inovovat
- nové učící algoritmy
- předzpracování dat
- nové struktury spojování neuronů
- například RNN (rekurentní neuronové sítě)
  - sítě byly dříve schopné vzít jen jednu hodnotu a na základě té predikovat další
  - pak přišel nápad propojit vzdálené vrstvy →
  - tím pádem předchozí informace ovlivňovaly informace v dalších vrstvách
  - síť poté dokázala vzít desítky - stovky hodnot a zohlednit je při predikci další
- self-attention mechanism (GPT) a mnoho dalších
- tokenizace (zpracování přirozeného jazyka)
- maticové výpočty
- vektorové výpočty



# Rychlejší...

- nové hardwarové prostředky
- například neuromorfní procesory
  - simulace neuronové sítě se prováděla virtuálně → síť fyzicky stále pracovala na CPU s tranzistory
  - pracuje s hodnotami 0/1
- neuroformní procesory fyzicky napodobují neurony a synapse
  - rozdíl je třeba práce se signály mezi 0-1
  - asynchronní fungování
- mnohonásobně rychlejší a úspornější oproti binárním procesorům

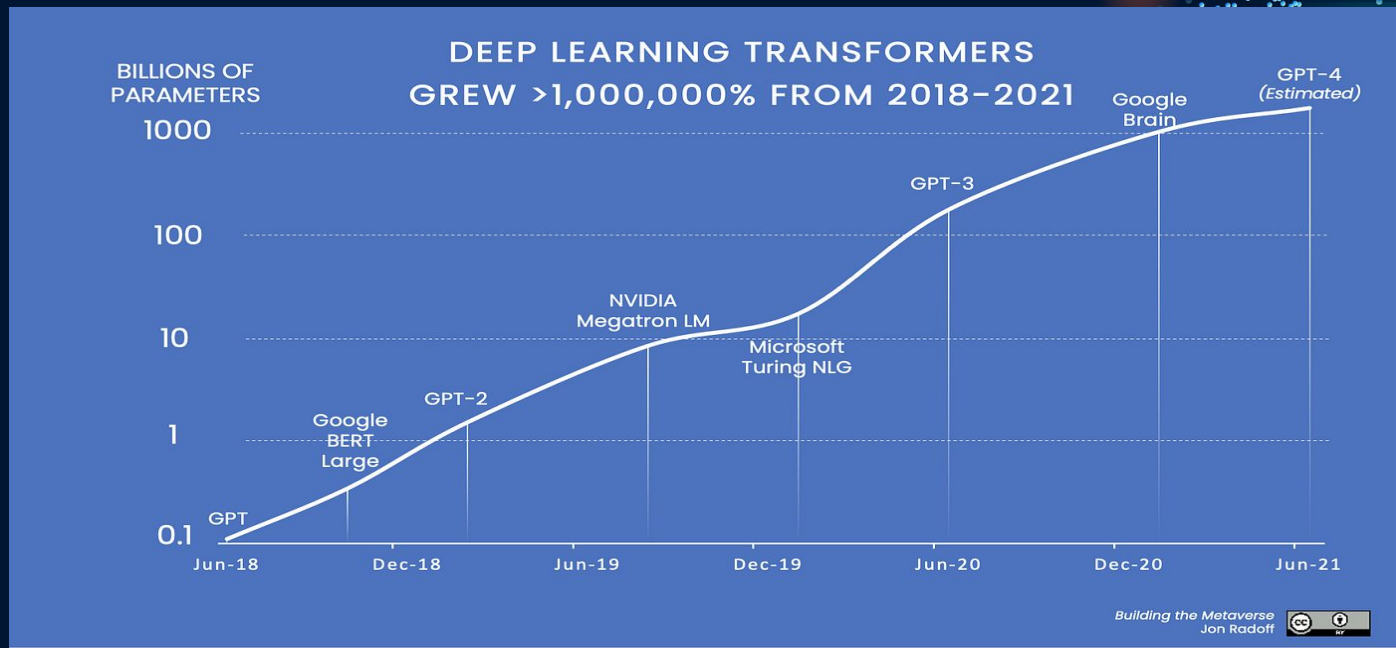


**Intel Loihi, 128 000 neuronů**

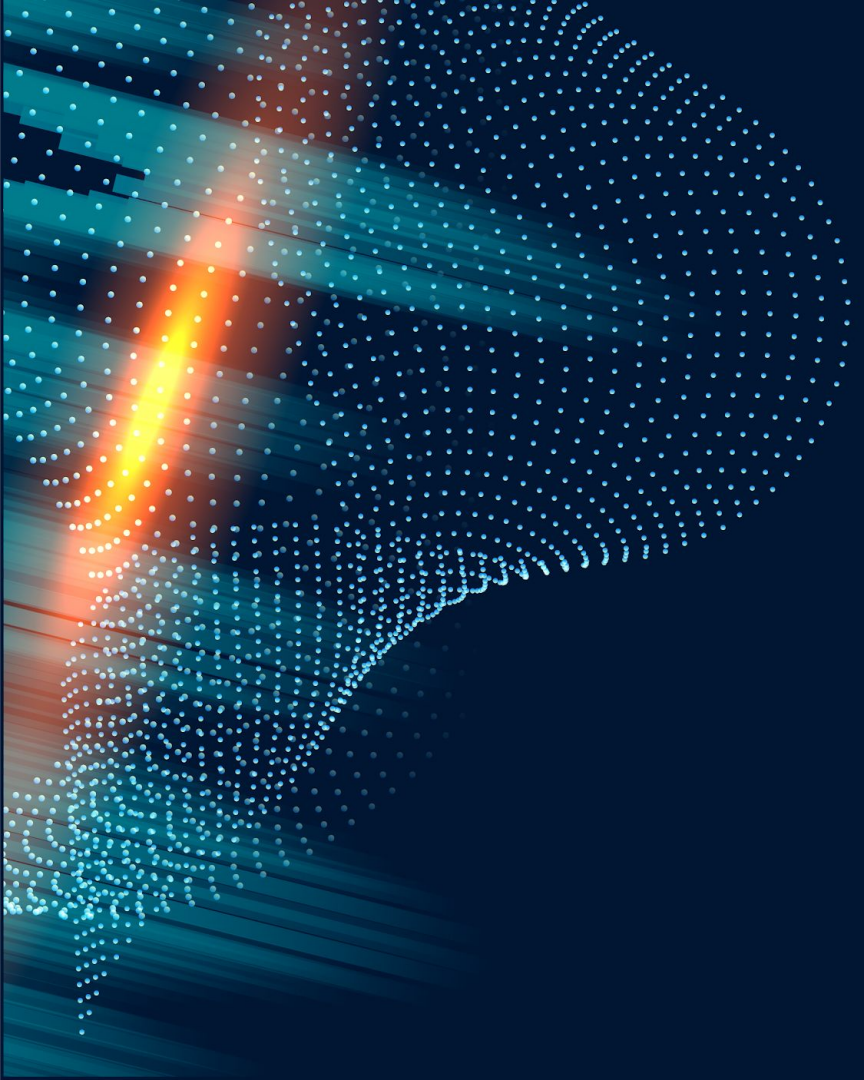


# Rozsáhlejší...

- závody v tom, kdo vytvoří větší (čtěte jako dražší/ náročnější) síť
- kdo ji zvládne nakrmit více daty
- se zlepšující se technologií můžou sítě nabývat čím dál tím oblodněších rozměrů







# Praktická ukázka



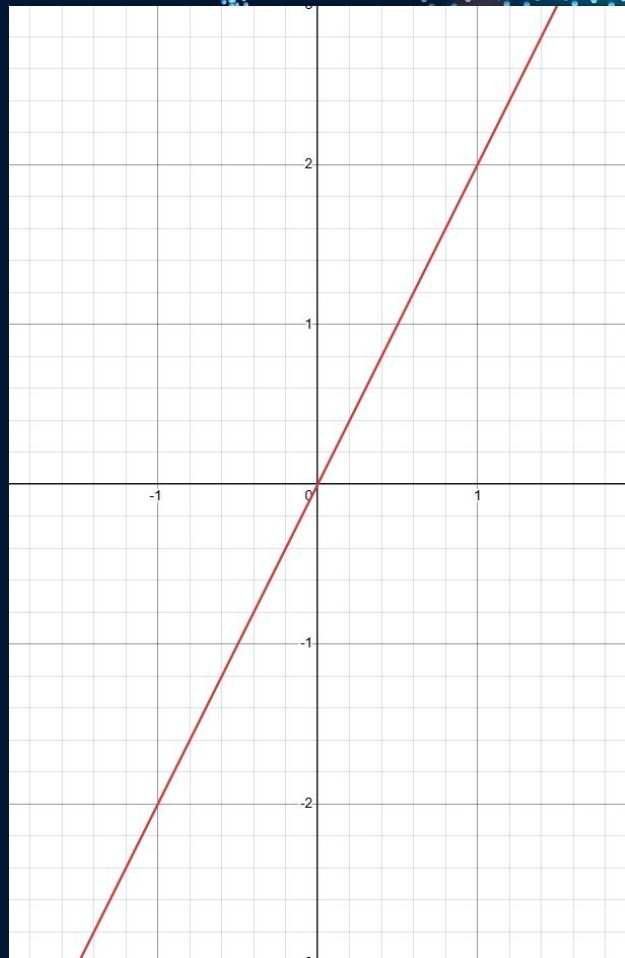
# Python

- programovací jazyk
- jeden z nejjednodušších
- želbohu jednoduchý na úkor výkonu



# Knihovny v Pythonu

- programátor nemusí všechno tvořit sám, může využít to, co vytvořili jiní
- například chceme vytvořit graf závislosti  $x$  na  $y$ 
  - pokud bychom nepoužili knihovny, python kód by měl desítky řádku
  - s použitím (hypotetických) knihoven stačí napsat: **`graf.zavislost(x(=1), y(=2))`**
    - knihovna může být napsaná v jiném jazyce → kód bude mnohem rychlejší
    - kód knihoven je mnohem propracovanější, stabilnější, výkonější
    - nevýhoda: nevíme, co se ve skutečnosti v kódu děje → známe jen výstup, nikoliv proces jakým vznikl
    - funguje jako “černá skříňka”



# Abstract

- MLP neuronová síť se vším všudy pro aproximaci vztahu mezi  $x, y$  v čistém pythonu
  - z kódu lze přesně vidět, co se děje
  - veškeré výpočty jsou prováděny manuálně, žádné knihovny
  - nejlepší pro detailní pochopení fungování neuronových sítí
  - na úkor výkonu a stability

# S knihovnamí a bez knihoven

- jádro sítě má nyní asi 80 řádků kódu
- s použitím knihoven měl kód asi 30 řádků a zrychlil přibližně 80x (díky využití výkonnějších programovacích jazyků a lepší optimalizace pro hardware (GPU))
- avšak kód knihovny má několik set řádků

```
63 training_data, labels = load_training_data("data.txt")
64
65 # Trénování modelu
66 model.fit(training_data, labels, epochs=100, batch_size=32)
```

```
174
175 def backward(x, y, layers):
176     global weights, biases
177     global errors
178
179     # Výpočet chyby jako MSE
180     errors = [(layers[-1][i] - y[i]) for i in range(OUTPUT_SIZE)]
181
182     # Gradient MSE: (2/N) * (y_pred - y_true)
183     mse_gradient = [(2 / OUTPUT_SIZE) * e for e in errors]
184     gradients = [mse_gradient] # Počáteční gradient chyby
185
186     # Backpropagation skrz vrstvy
187     for i in range(NUM_HIDDEN_LAYERS, -1, -1):
188         layer_errors = [sum(gradients[0][j] * weights[i][k][j] for j in range(len(gradients[0])))
189                         for k in range(len(layers[i]))]
190         gradients.insert(0, [layer_errors[k] * leaky_relu_derivative(layers[i][k])
191                             for k in range(len(layer_errors))])
192
193     # Úprava vah a biasů
194     for j in range(len(gradients[1])):
195         for k in range(len(layers[1])):
196             weights[1][k][j] -= LEARNING_RATE * gradients[1][j] * layers[1][k]
197             biases[1][j] -= LEARNING_RATE * gradients[1][j]
```

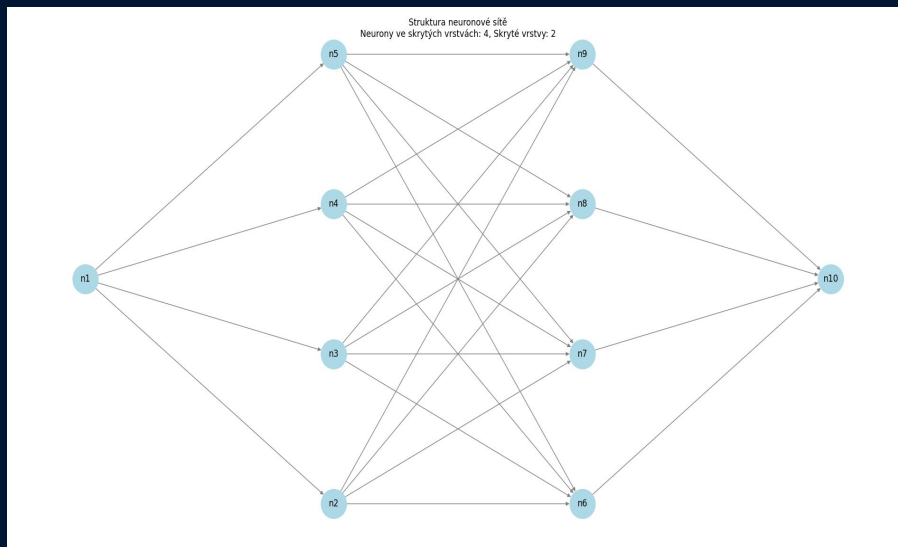


# Model

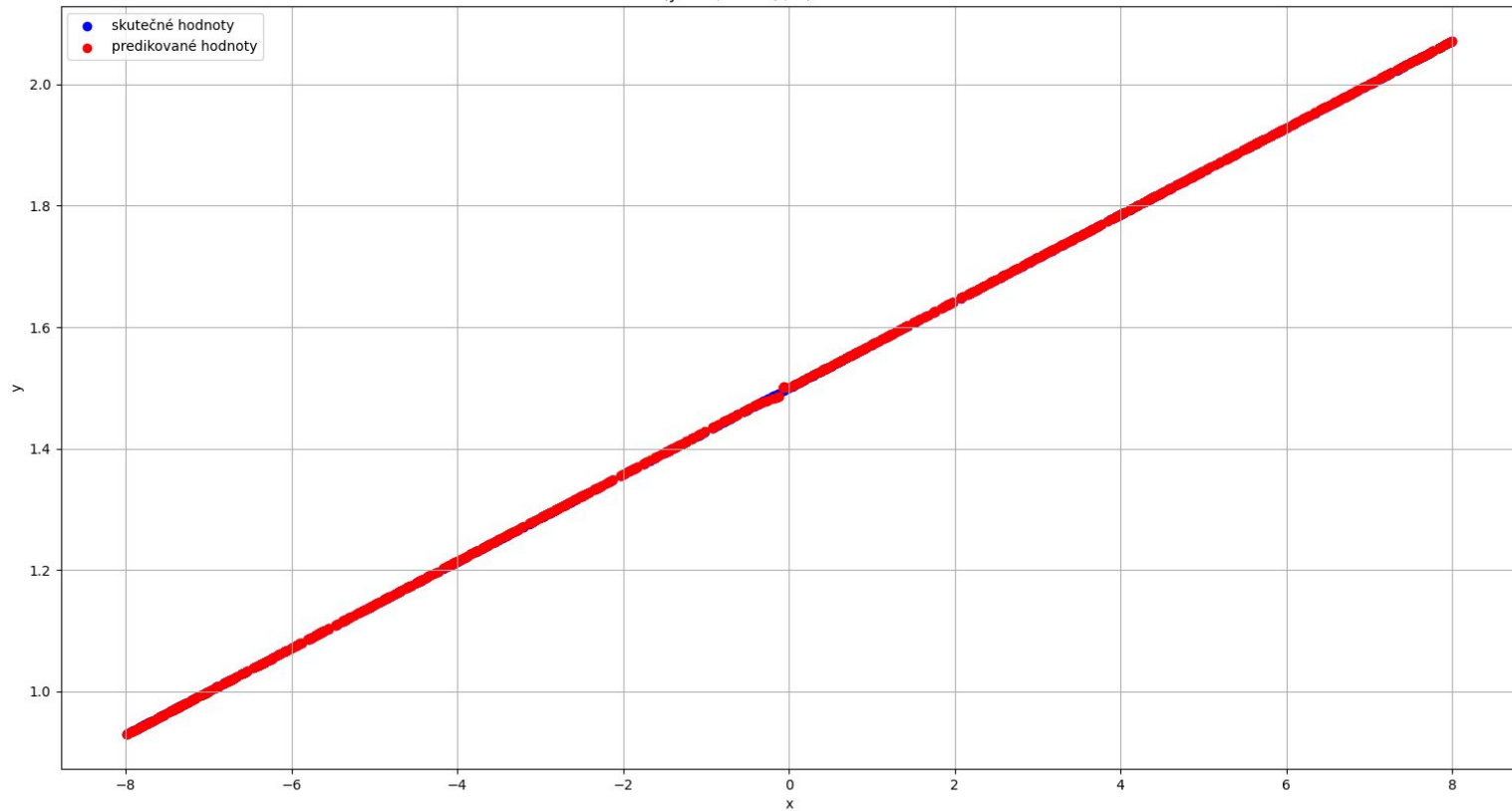
- 1 vstupní neuron
- nastavitelný počet skrytých vrstev
- nastavitelný počet neuronů ve skrytých vrstvách
- struktura, chyba a rychlost učení modelu je vizuálně reprezentována grafy
  - na grafy jsem kvůli úspoře času použil knihovny
- 1 výstupní neuron
- síť trénována na 1000 náhodných příkladech v rozsahu -8 8 (kvůli stabilitě)
- testována na odlišných 1000 náhodných příkladech
- maximum neuronů je kvůli času testování 128
- v rámci pokusů jsem došel k závěru že nejstabilnější a nejreprezentativnější jsou modely s:
  - 2 nebo 4 skrytými vrstvami
  - 4 nebo 8 nebo 16 nebo 32 neurony v každé skryté vrstvě
- obecně platí:
  - větší síť = nestabilnější
  - čím větší síť, tím složitější vztah zvládne

# Lineární příklad $(x / 7 + 3) / 2$

- i nejmenší síť zvládla téměř přesnou aproximaci (stačila by i jedna skrytá vrstva)
- testy větších modelů nejsou potřeba



porovnání skutečné a naučené funkce  
neurony ve skrytých vrstvách: 4, skryté vrstvy: 2, alpha: 0.1, learning\_rate: 0.01  
funkce:  $(y = x / 7 + 3) / 2$ , rozsah train dat: -8 8



### Křivka učení

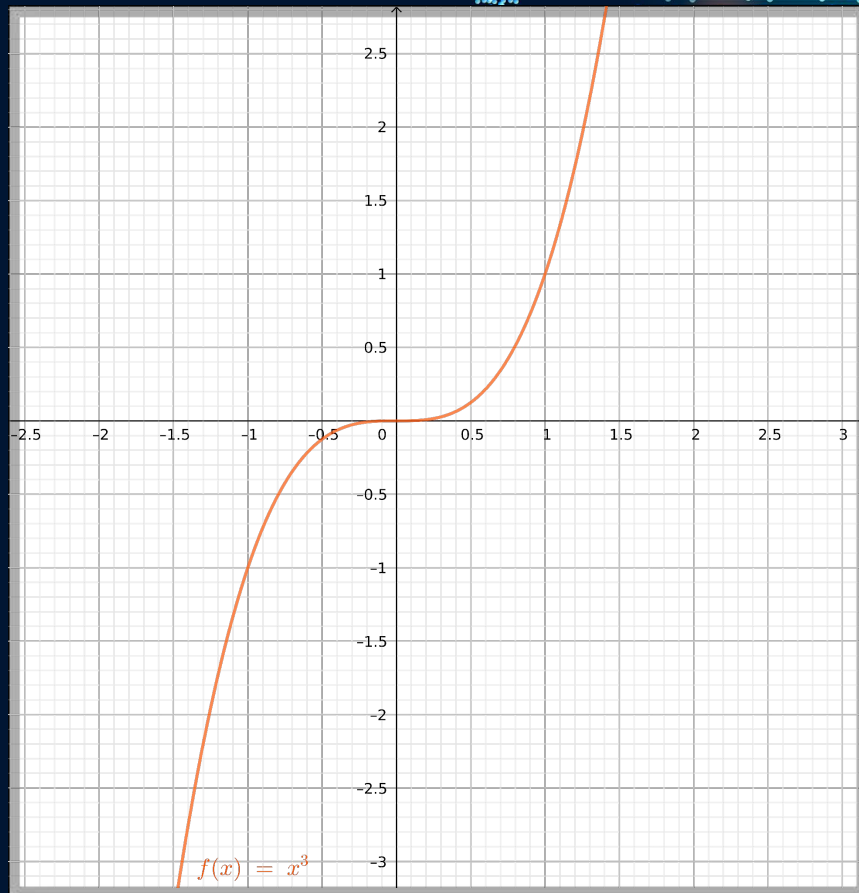
čas na epochu: 0.01796, neurony ve skrytych vrstvách: 4, skryté vrstvy: 2, alpha: 0.1, learning\_rate: 0.01

funkce:  $(y = x / 7 + 3) / 2$ , rozsah train dat: -8 8



# Třetí mocnina

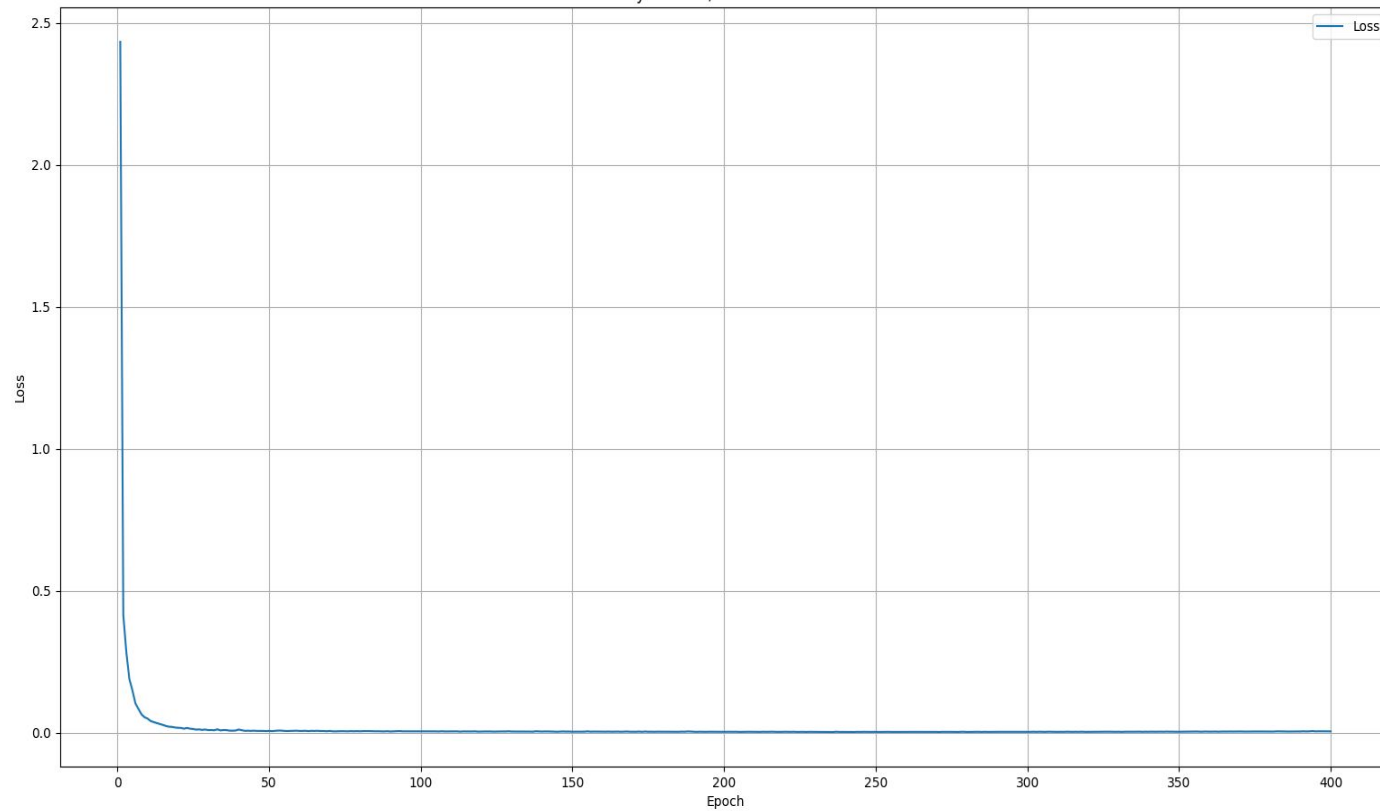
- nejmenší model nestačil → zvýšení počtu neuronů ve vrstvách na 8
- (n8-v2)



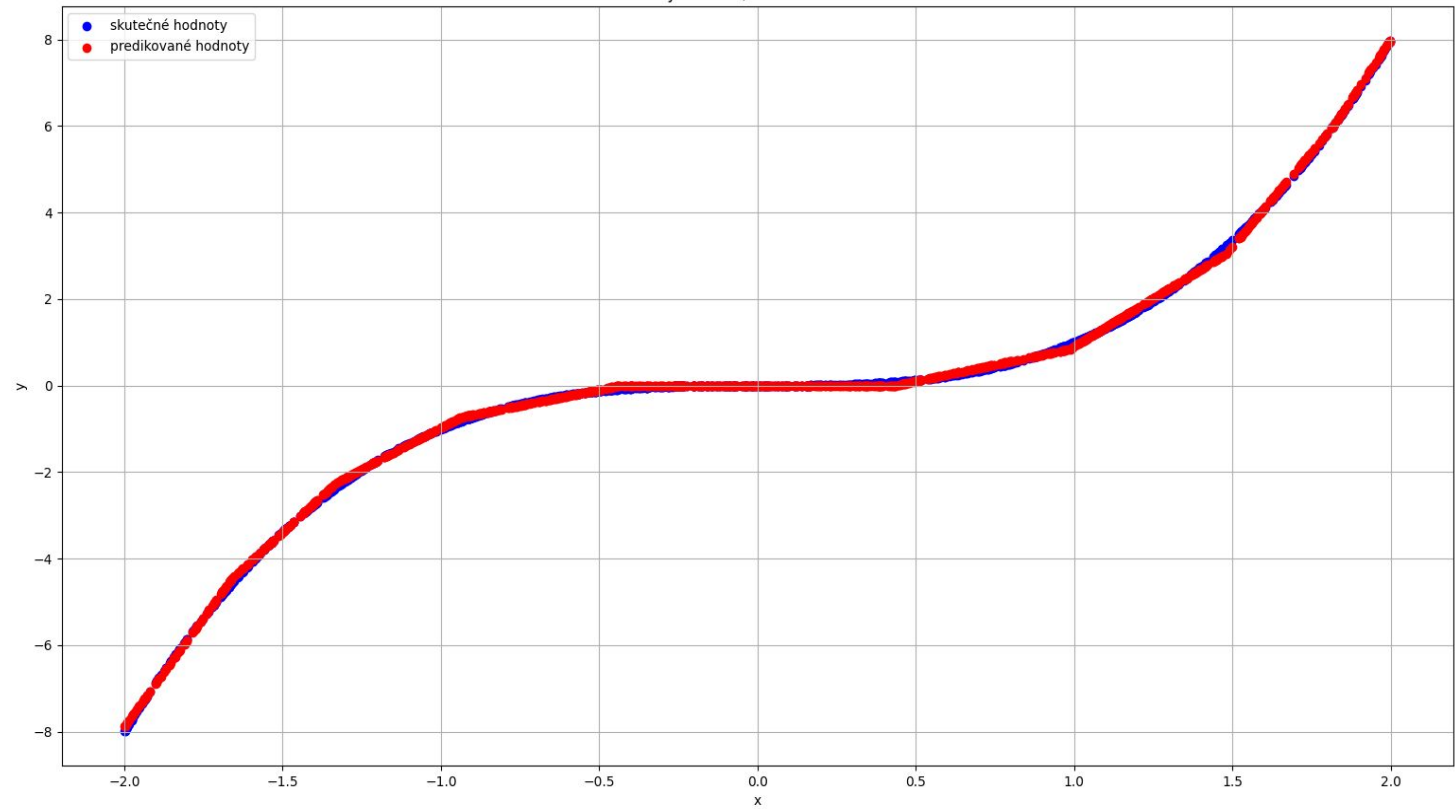


### Křivka učení

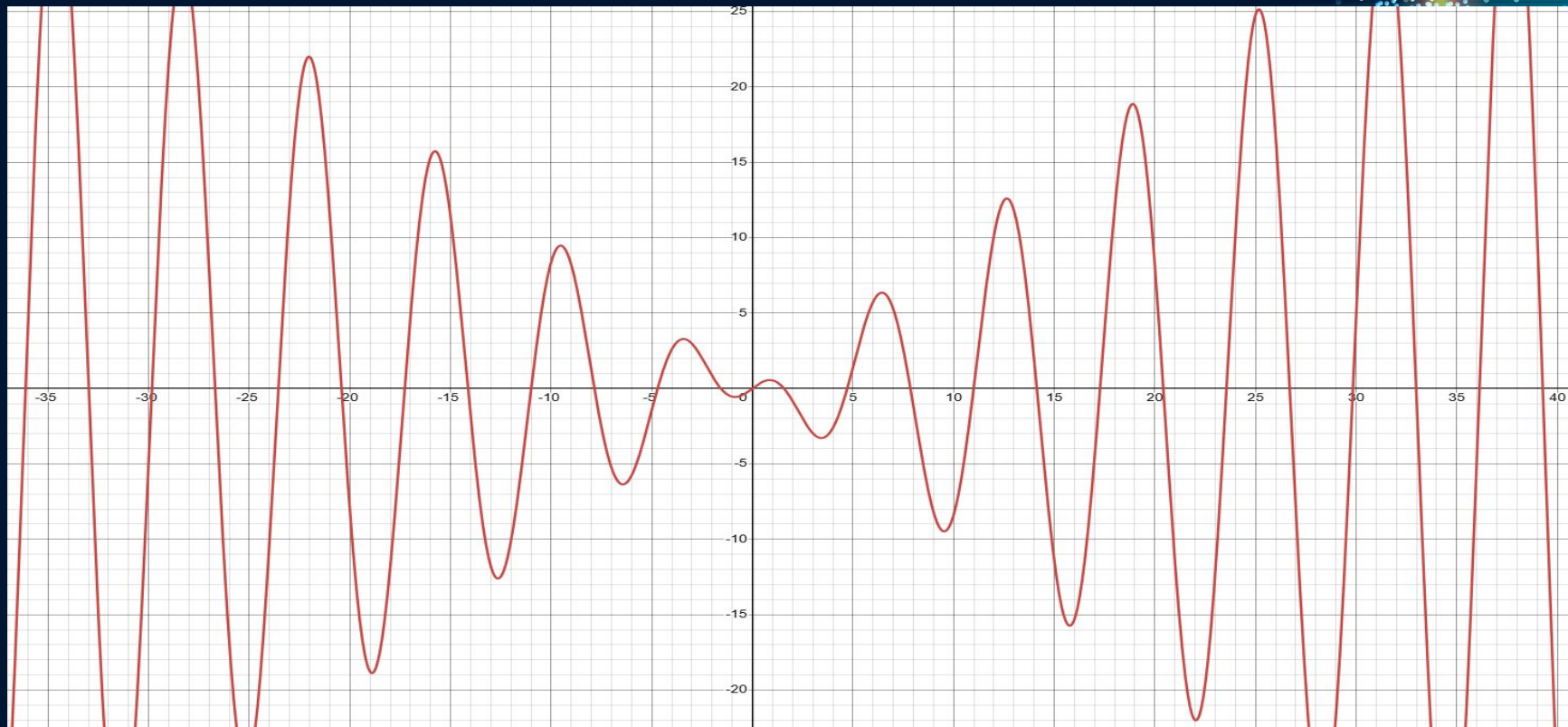
čas na epochu: 0.0362, neurony ve skrytych vrstvách: 8, skryté vrstvy: 2, alpha: 0.001, learning\_rate: 0.001  
funkce:  $y = x^3$ , rozsah train dat: -8 8



porovnání skutečné a naučené funkce  
neurony ve skrytých vrstvách: 8, skryté vrstvy: 2, alpha: 0.001, learning\_rate: 0.001  
funkce:  $y = x^3$ , rozsah train dat: -8 8

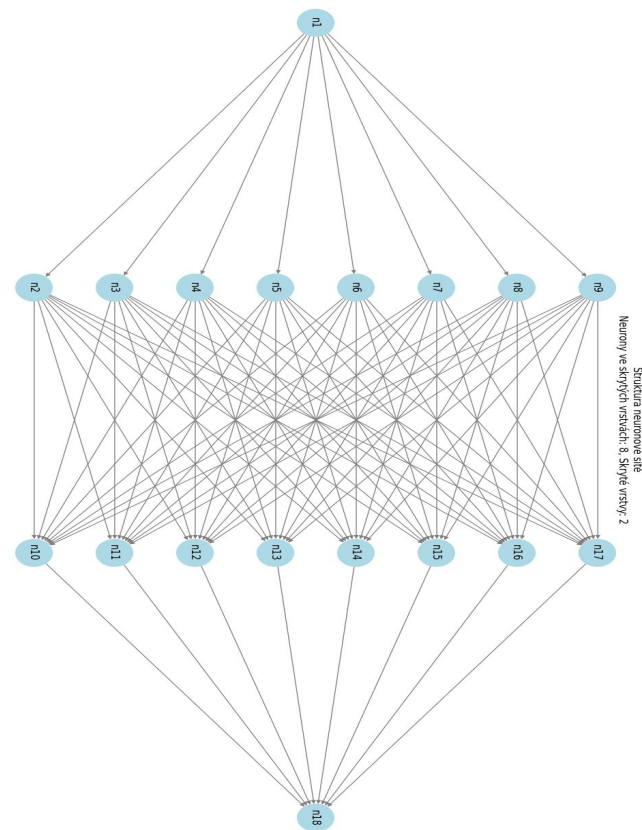


$$y = x * \cos(x)$$



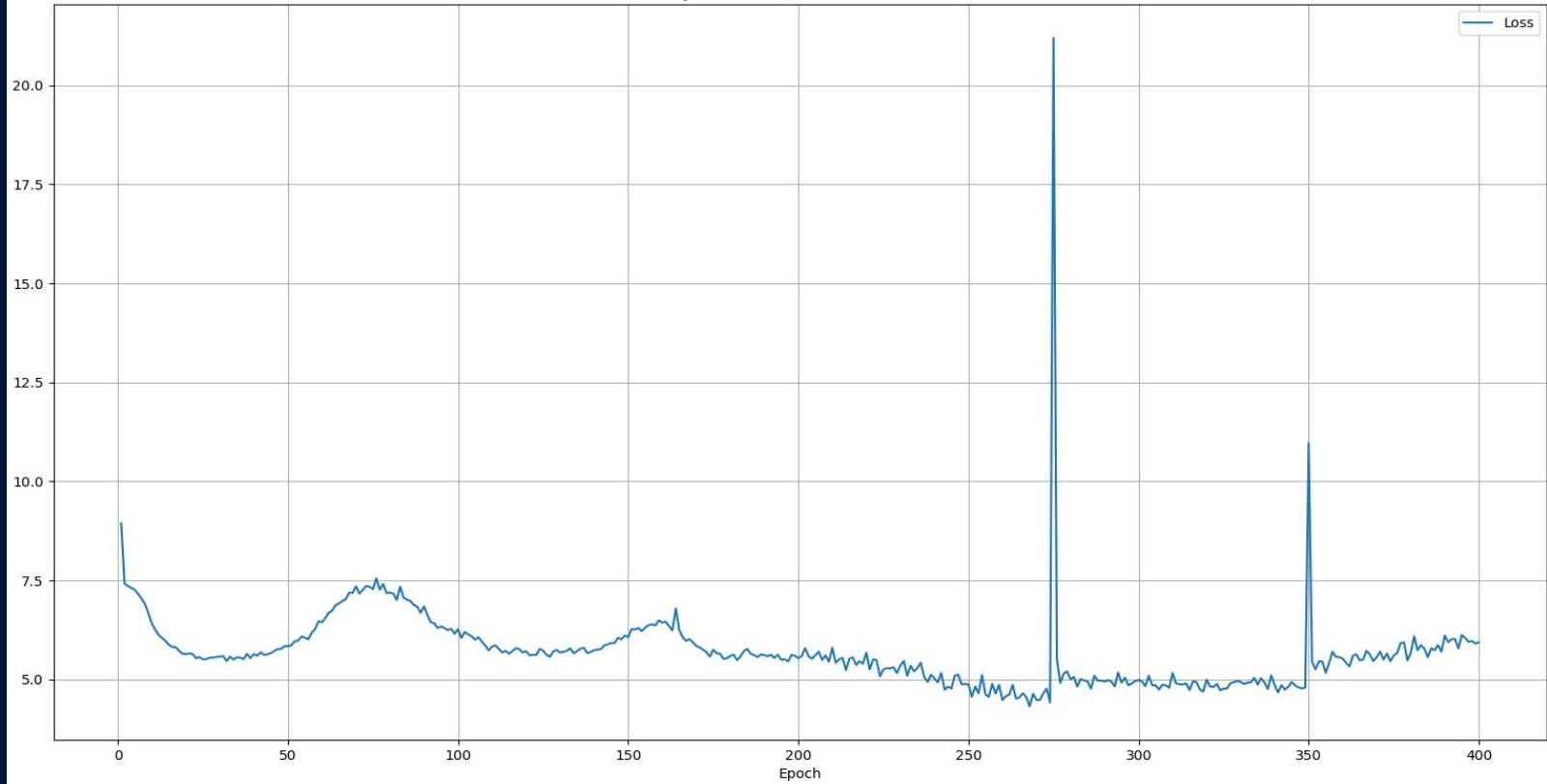
$$y = x * \cos(x)$$

- síť n8-v2 nebyla dostatečná
- vztah je nejspíš příliš komplexní pro toto množství vrstev → síť nemá dostatečnou “hloubku”



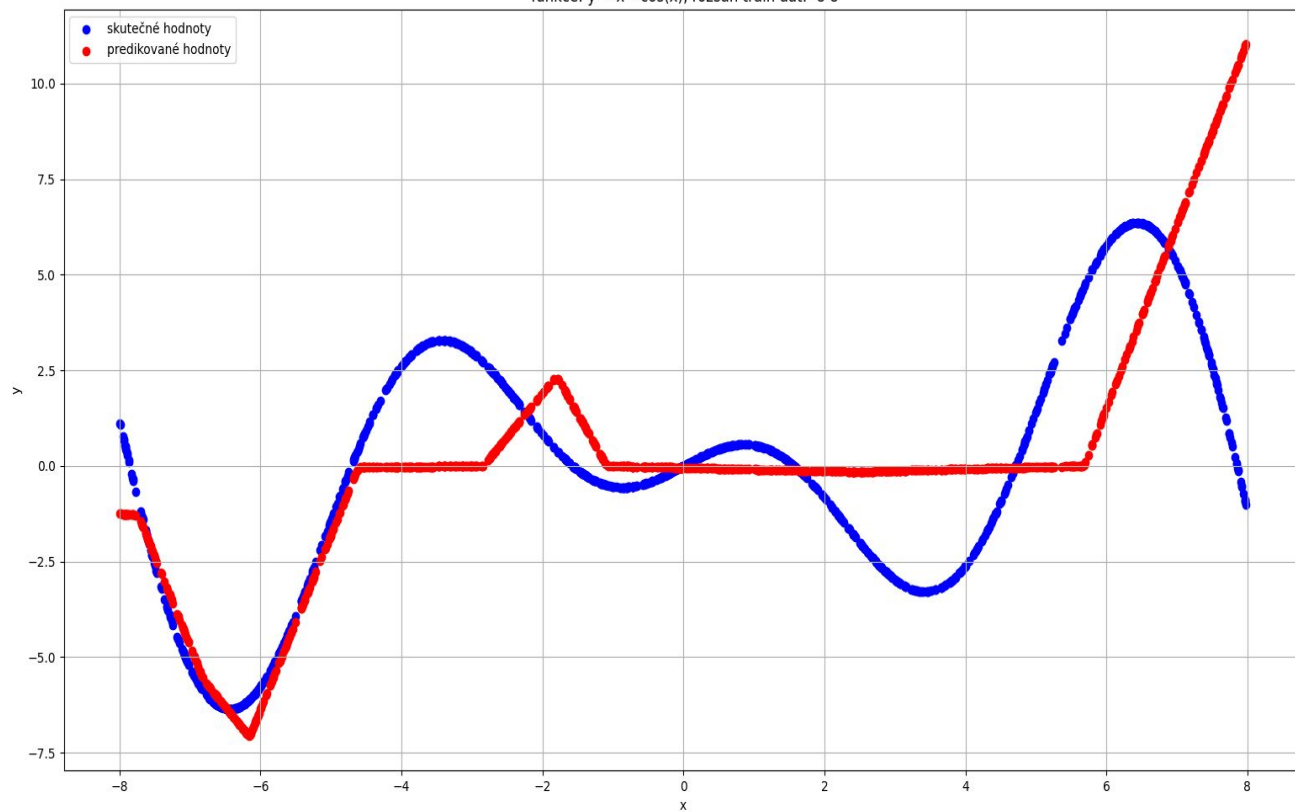
### Křivka učení

čas na epochu: 0.0347, neurony ve skrytych vrstvách: 8, skryté vrstvy: 2, alpha: 0.01, learning\_rate: 0.0001  
funkce:  $y = x * \cos(x)$ , rozsah train dat: -8 8



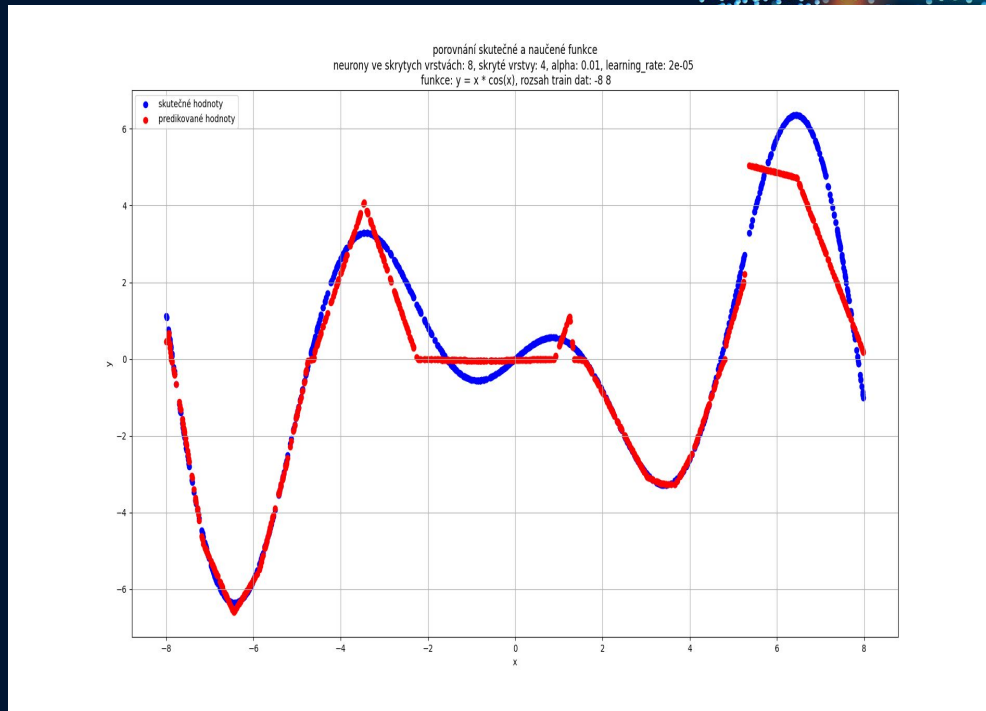


porovnání skutečné a naučené funkce  
neurony ve skrytých vrstvách: 8, skryté vrstvy: 2, alpha: 0.01, learning\_rate: 0.0001  
funkce:  $y = x * \cos(x)$ , rozsah train dat: -8 8



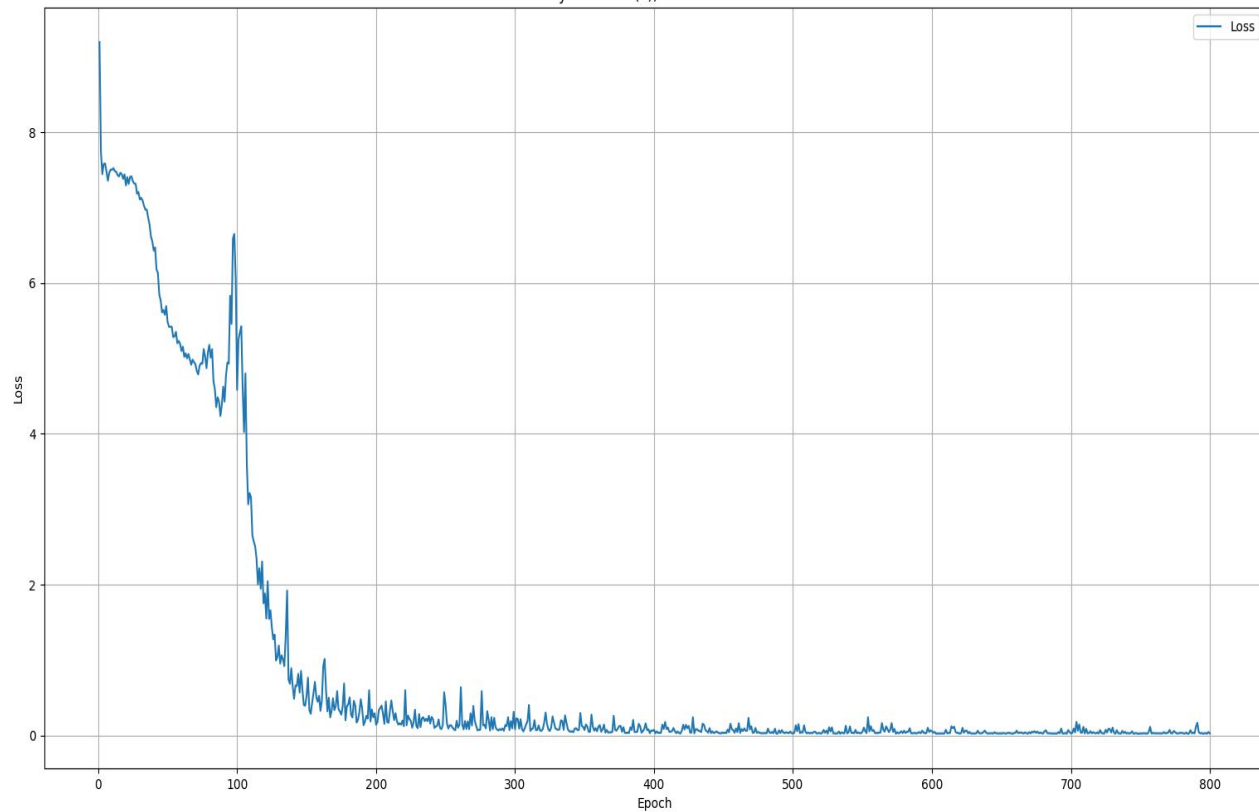
$$y = x * \cos(x)$$

- n8-v4, snížení rychlosti učení, více epoch →
- n16-v4 → podobný výsledek
- n32-v4 → cca 12 minut tréninku
  - s úpravou parametrů a 800 epochami

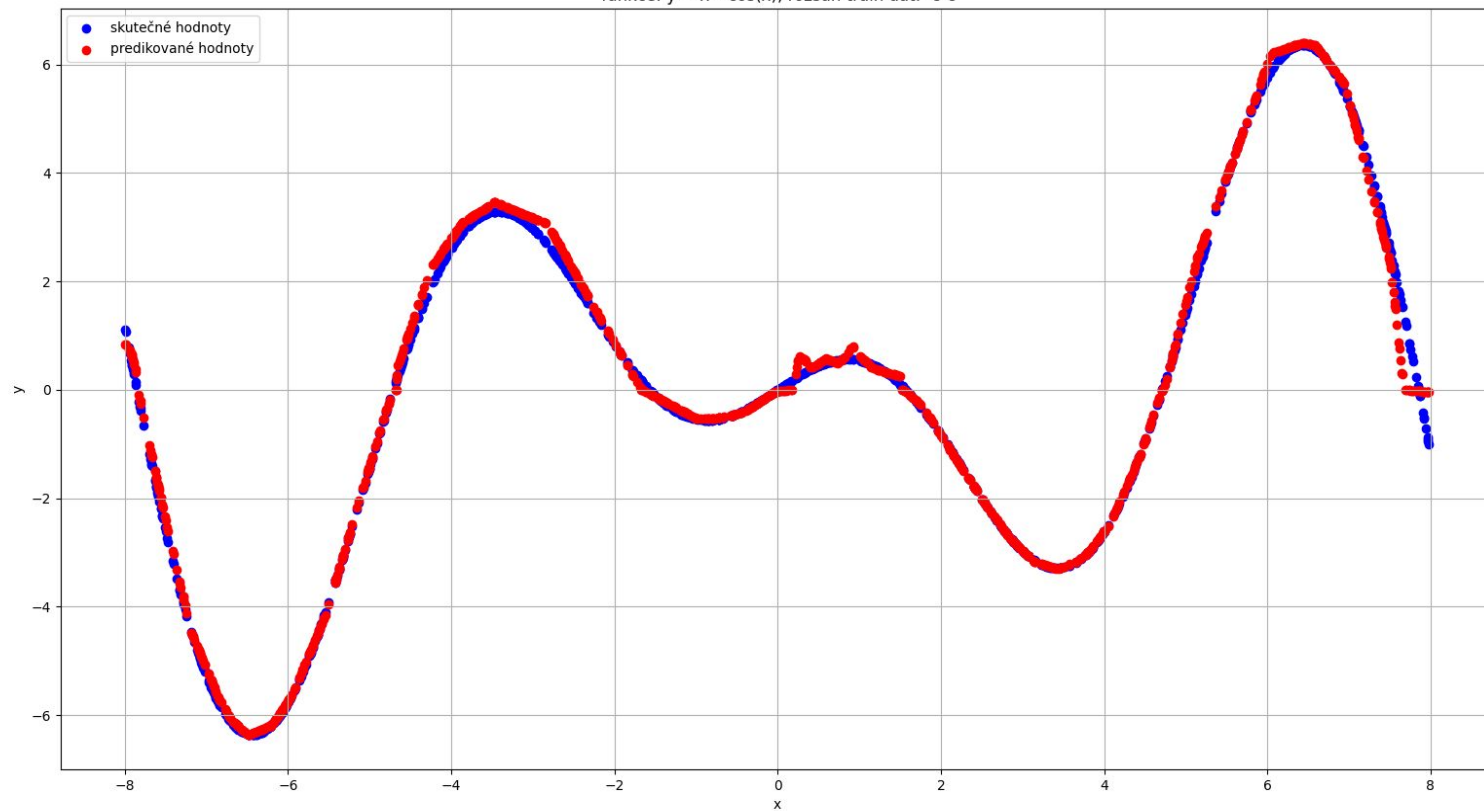


### Křivka učení

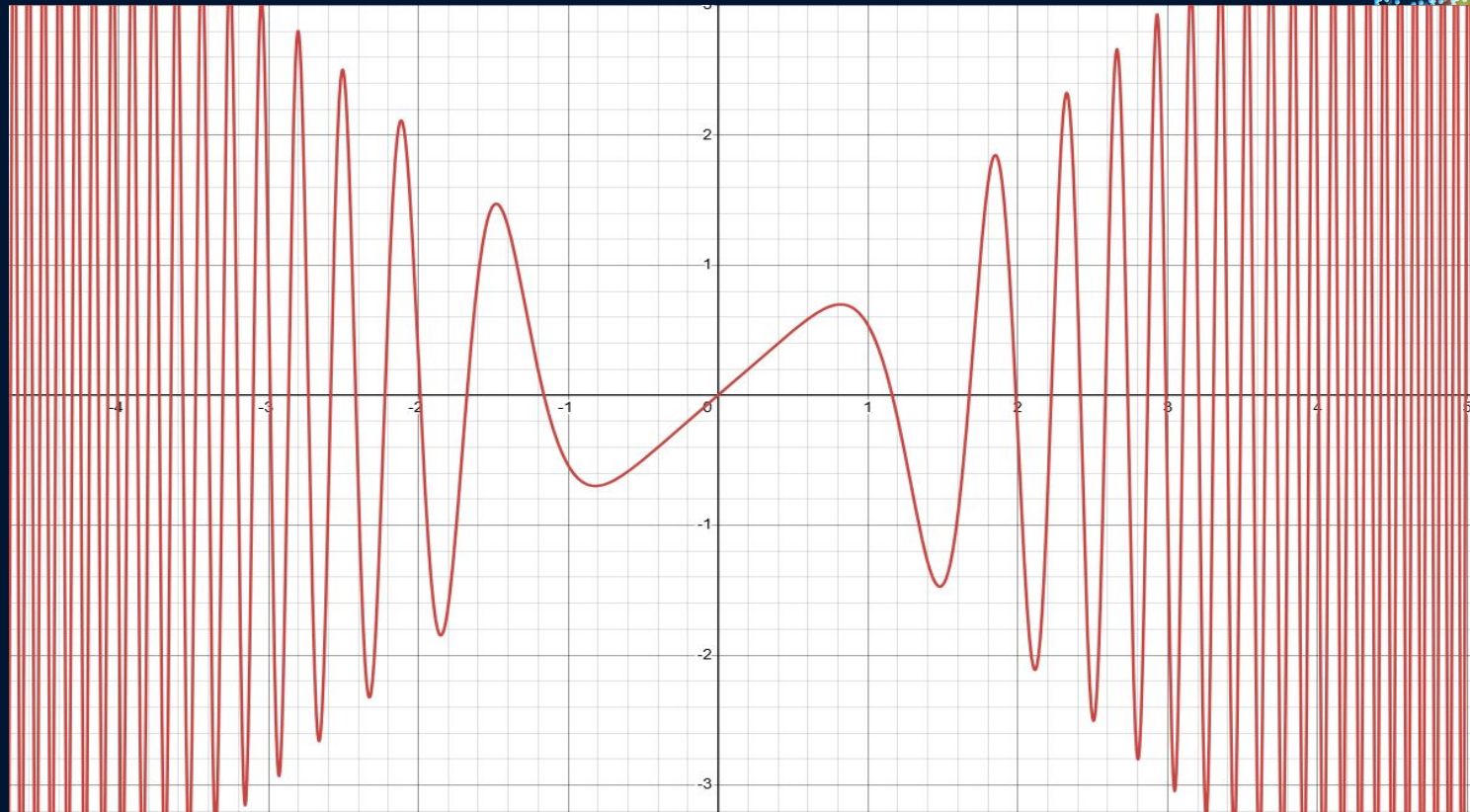
čas na epochu: 0.7456, neurony ve skrytych vrstvách: 32, skryté vrstvy: 4, alpha: 0.01, learning\_rate: 1e-05  
funkce:  $y = x * \cos(x)$ , rozsah train dat: -8 8



porovnání skutečné a naučené funkce  
neurony ve skrytých vrstvách: 32, skryté vrstvy: 4, alpha: 0.01, learning\_rate: 1e-05  
funkce:  $y = x * \cos(x)$ , rozsah train dat: -8 8



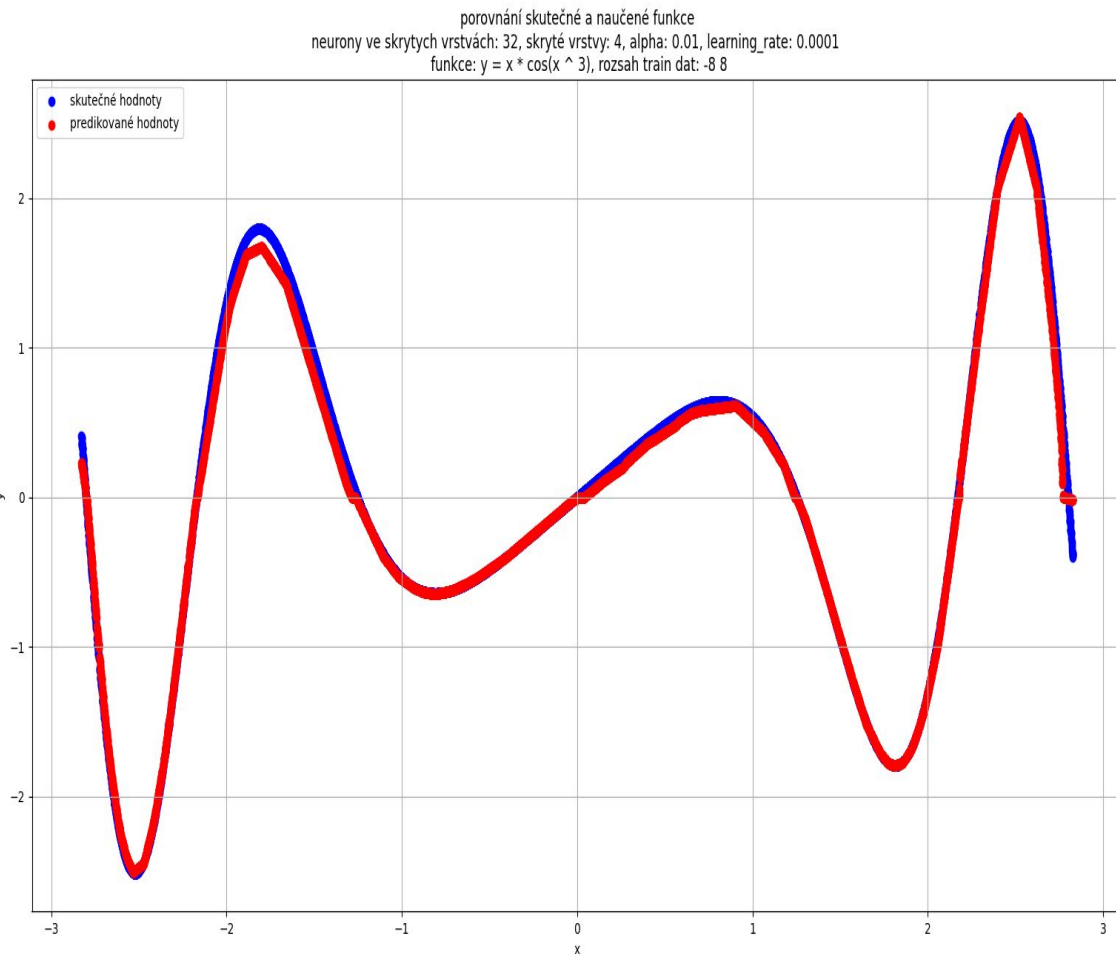
$$y = x * \cos(x \wedge 2)$$





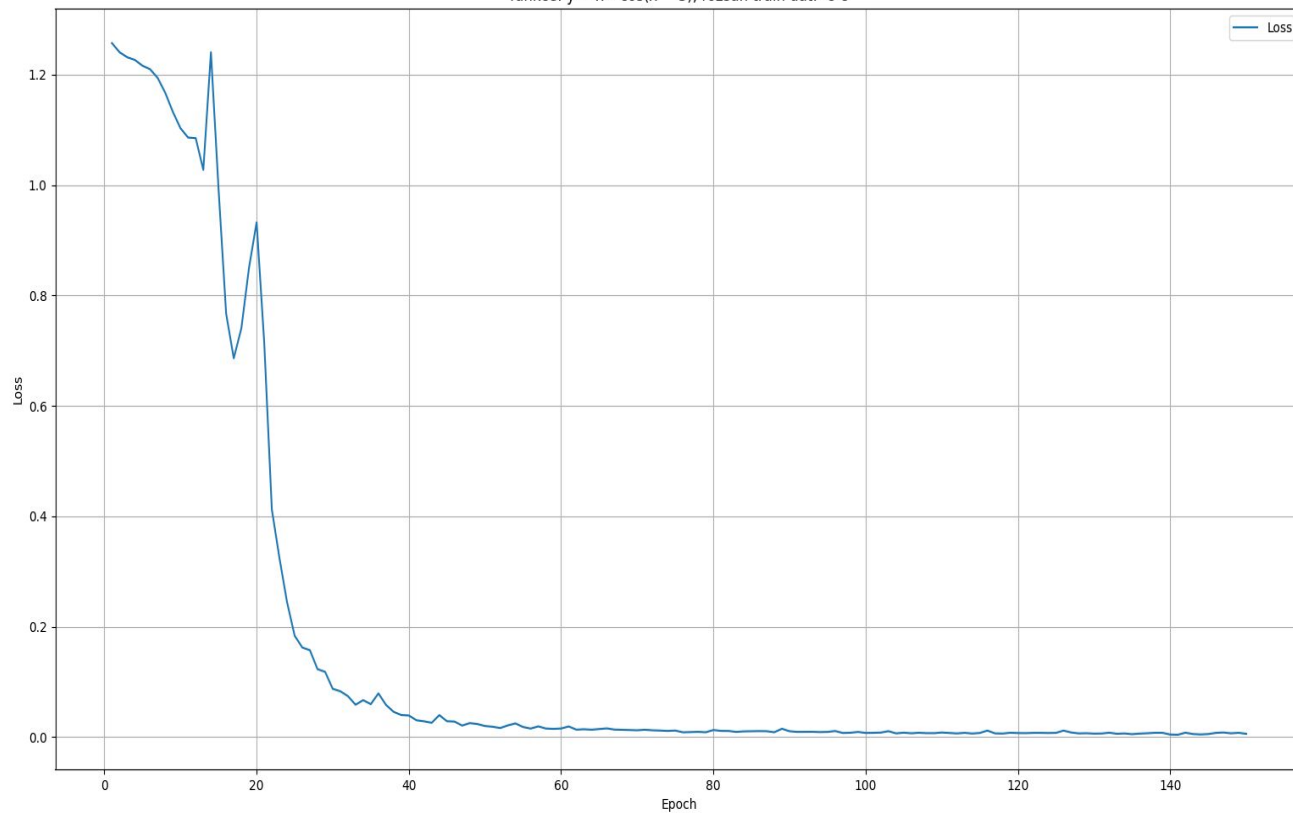
$$y = x * \cos(x \wedge 2)$$

- n32-v4 → 10 000 příkladů →
  - 18 minut tréninku
- ostatní modely selhaly
- při menším počtu příkladů selhaly všechny modely

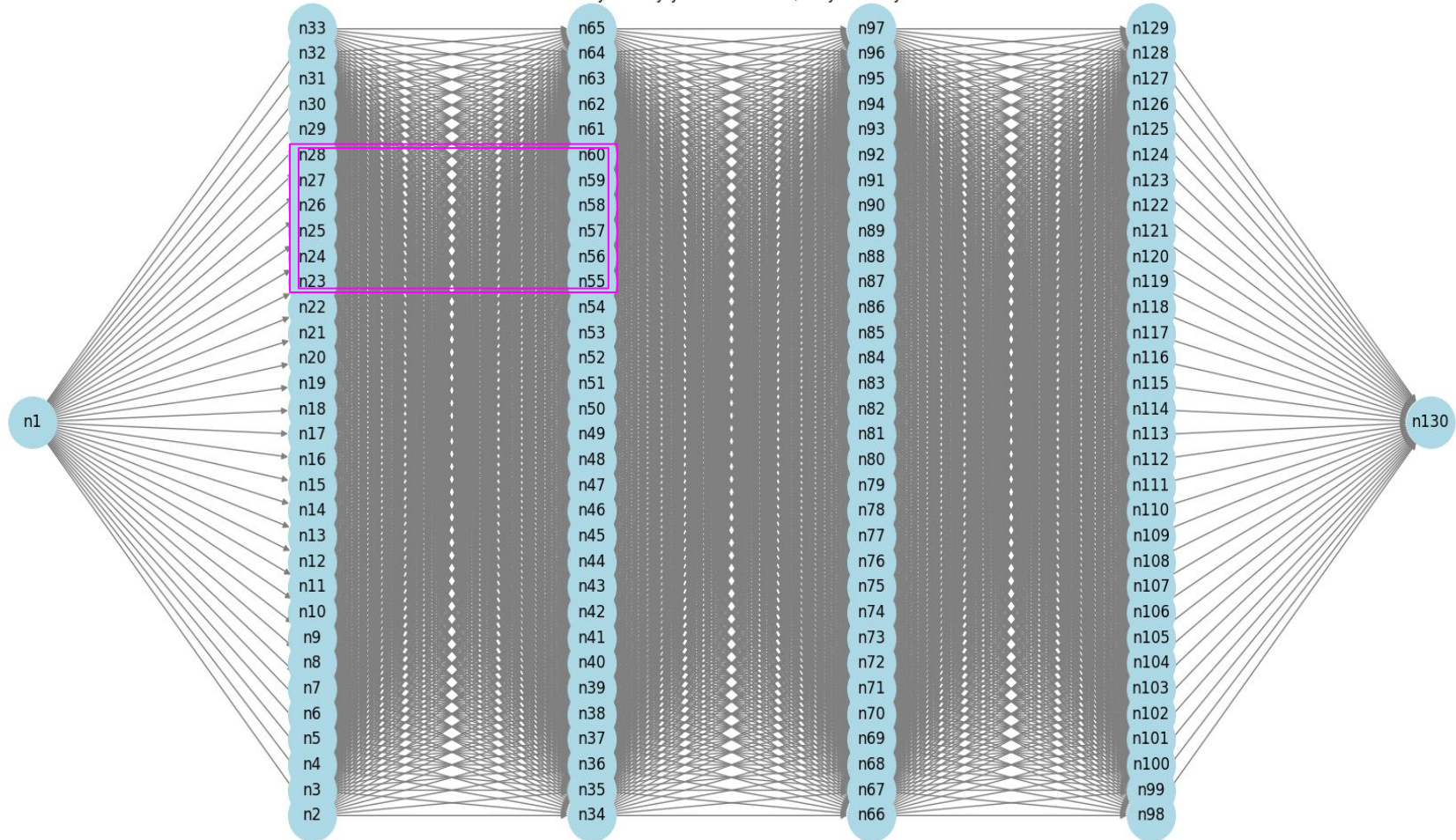


### Křivka učení

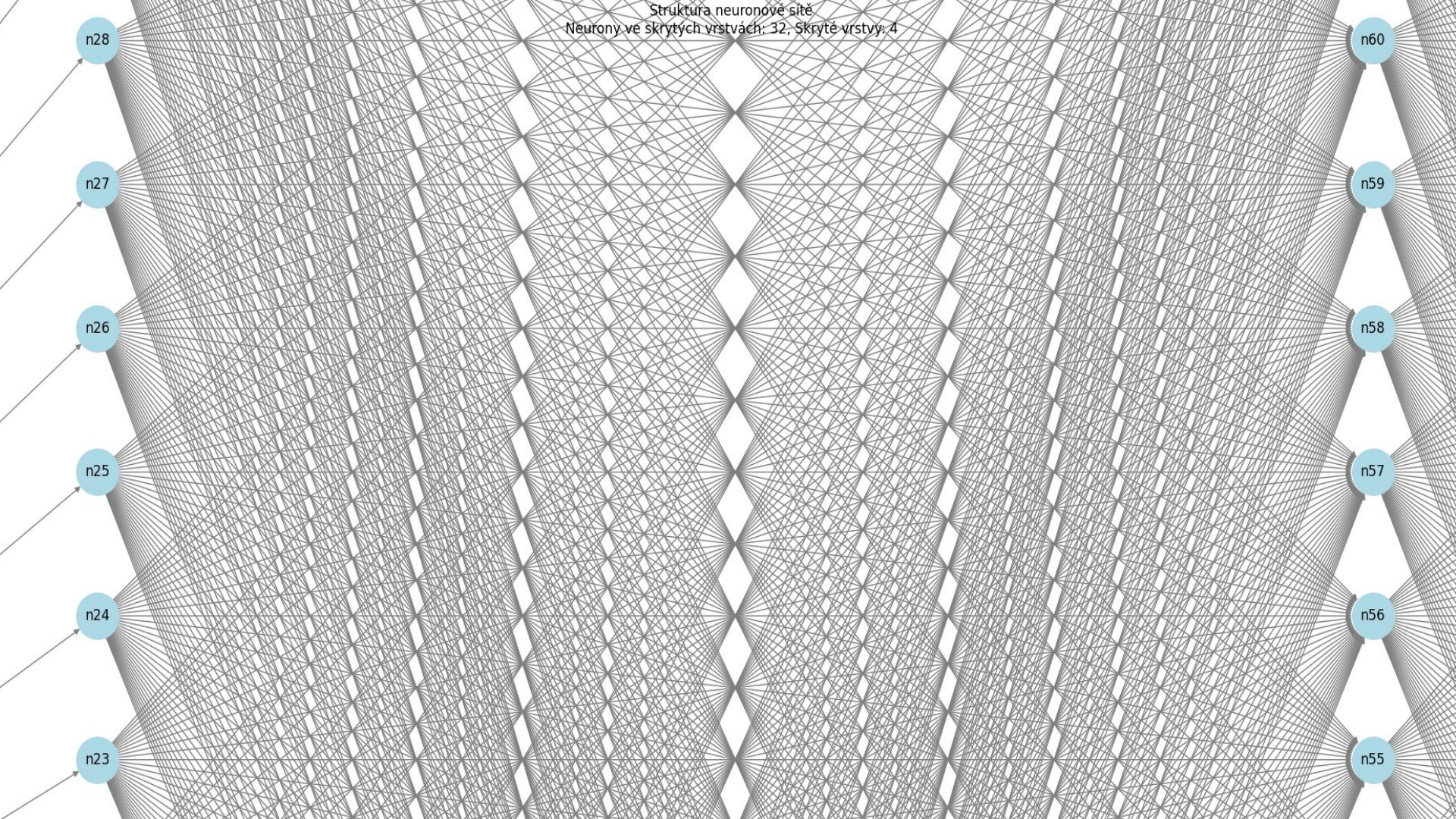
čas na epochu: 7.2761, neurony ve skrytych vrstvách: 32, skryté vrstvy: 4, alpha: 0.01, learning\_rate: 0.0001  
funkce:  $y = x * \cos(x^3)$ , rozsah train dat: -8 8



Struktura neuronové sítě  
Neurony ve skrytých vrstvách: 32, Skryté vrstvy: 4

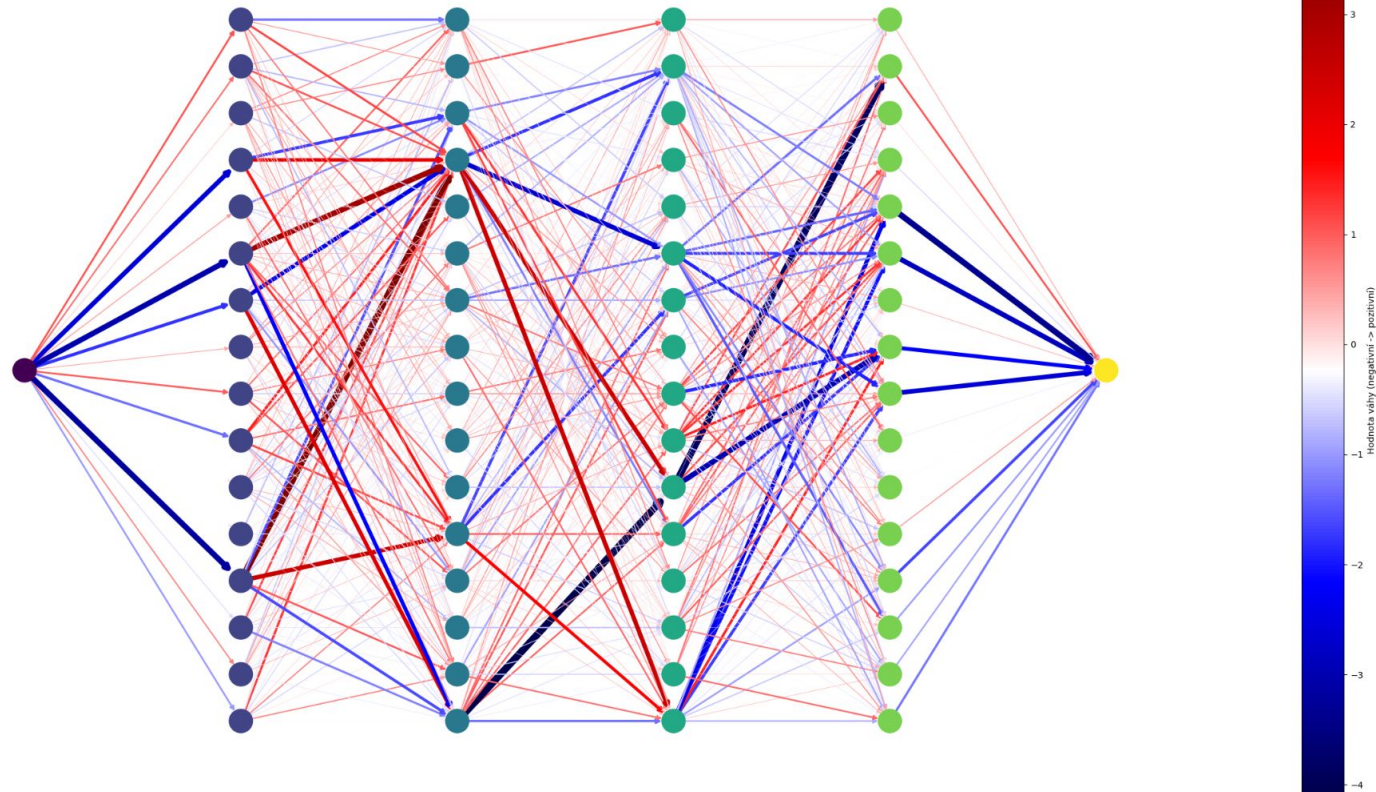






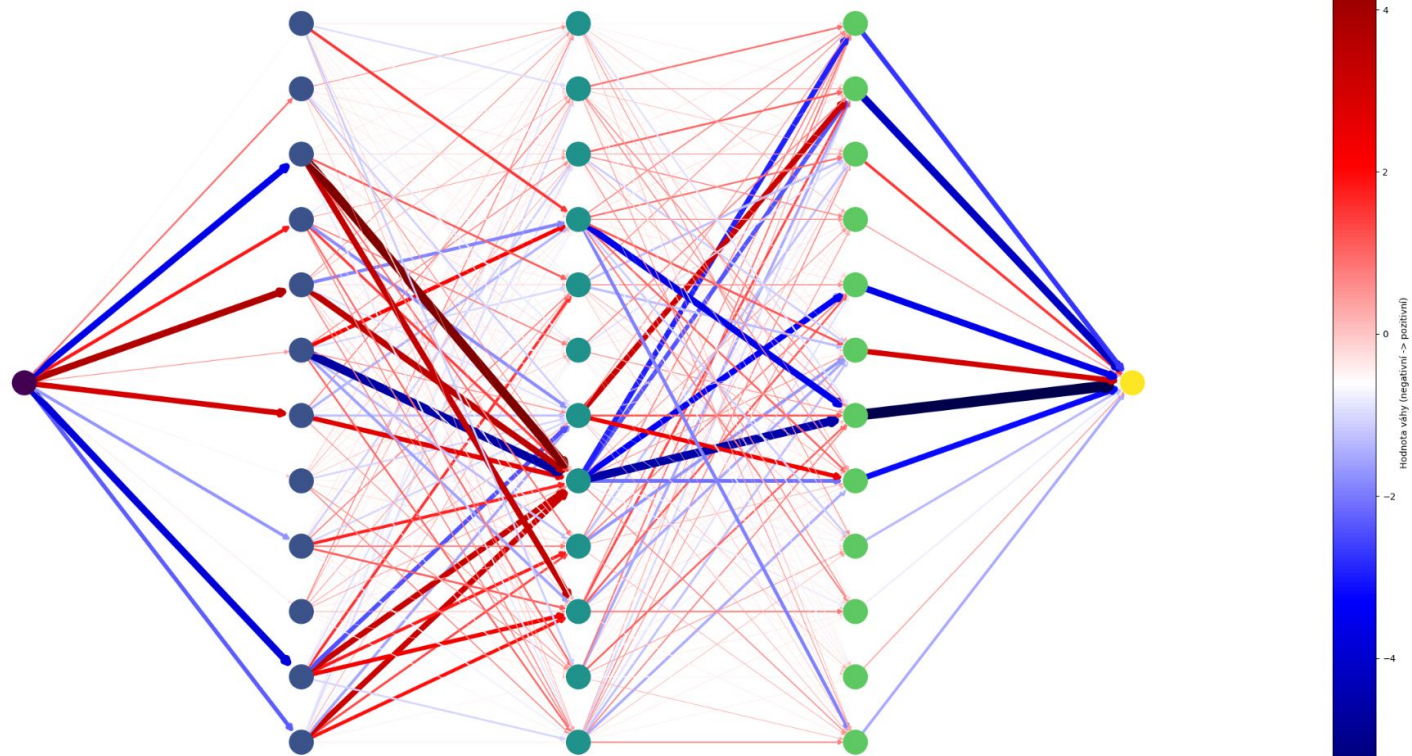


# Heatmap (síla synapsí)

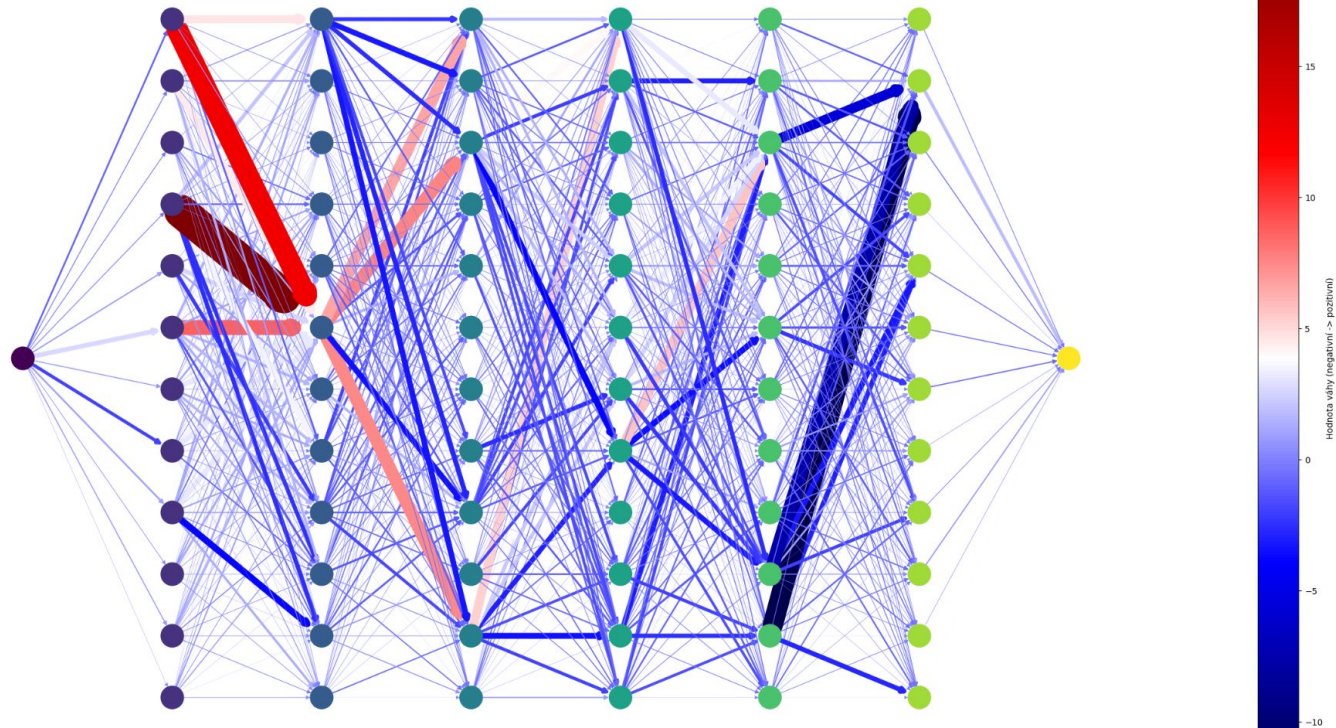




# Heatmap (síla synapsí)



# Heatmap (sila synapsí)



# Dodatky k modelu

- při delším tréninku a větším výkonu hardware by zvládl i obtížnější úkony
- klasifikaci ručně psaných číslic z MNIST zvládl taktéž, avšak:
  - kvůli nutnému zrychlení → akcelerace gpu\* pomocí knihoven
  - použití knihoven pro práci s daty
  - nutná změna architektury kvůli změně úlohy
  - nutné použití maticových výpočtů
- veškeré materiály a kódy jsou (měly by být) k dispozici na [https://github.com/jeohan19/inf\\_MLP](https://github.com/jeohan19/inf_MLP)
- grafy byly generovány programem během testování
- při přepsání kódu pro výpočty na GPU (RX 6800 16GB vram): zrychlení 40x - 80x oproti výpočtům na Ryzen 7600
- \* výkonnostní rozdíl cpu vs gpu → počet jader (max 128 vs max cca 20 000 )
  - gpu je lepší pro zpracování velkého množství nenáročných operací



# Čistě pro zajímavost...(data nejsou vždy přesná)

- GPT 2:
  - 48 vrstev
  - 1600 neuronů v každé
  - 1 500 000 000 parametrů
  - trénování na 40 GB textu
  - trénink cca 40 000 - 100 000 dolarů
- GPT 3:
  - 175 000 000 000 parametrů
  - asi 570 GB
- GPT 4:
  - cca 1,7 bilionu parametrů (cca 1000x více než GPT2)
  - tréninková data mnohonásobně větší než GPT 3 (mimo jiné zahrnují nejen text, jako u předchozích modelů, ale i fotografie)

# Přesnost informací ohledně modelů

- z části odhady na základě různých informací
- OpenAI kompletně zveřejnilo pouze model GPT 2
- u ostatních zveřejnilo jen některé informace (počet parametrů, velikost tréninkových dat v tokenech (číselná reprezentace slov) a podobně)



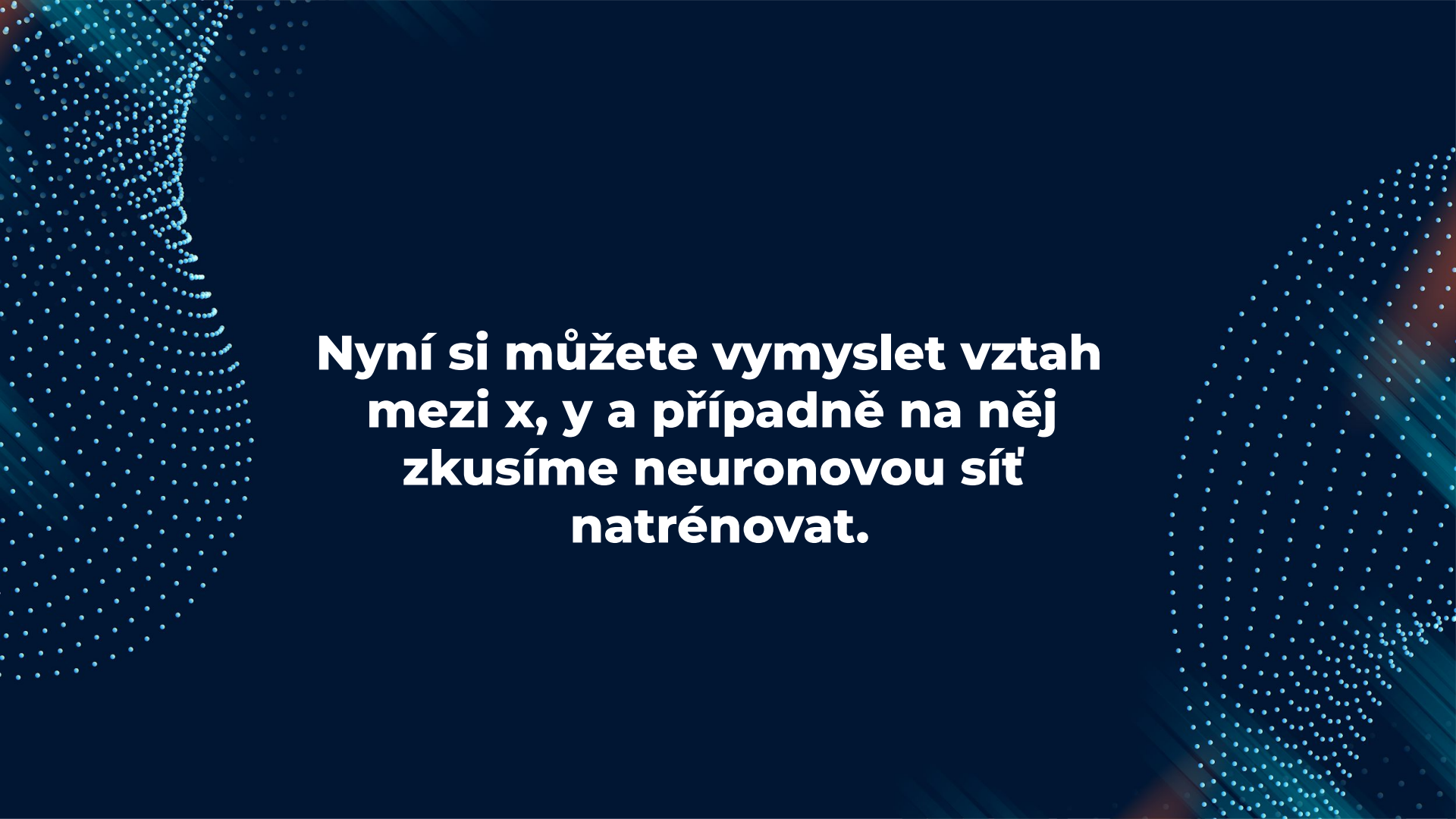
# HPptxt (Harry Potter plain text)

- jednotka velikosti textových souborů
- jednoduše představitelná
- všech 7 knih Harry Potter v angličtině v .txt souboru
- cca 6.3 MB
- 40 GB = cca 40 000 MB = cca 6 350 HPptxt
- celá anglická wikipedie v .txt asi 100 GB



# Náročnost sítí

- i největší modely (GPT 4) generují data více méně rychlostí, jakou jsme schopni číst
- GPT 4 → až 1000x větší než GPT 2
- zdrojové kódy a natrénované hodnoty GPT 2 byly zveřejněny
- po stažení a zprovoznění na školních počítačích \* → generování jedné odpovědi trvalo 10 - 15 minut\* (trénink by trval mnoho let)
  - \*při použití GPU RX 6800 → 20 - 40 sec
- \* instalace žádného softwaru neproběhla (python knihovny a podobně), GPT byla spuštěna z flash disku za pomoci venv (python virtuální prostředí)



**Nyní si můžete vymyslet vztah  
mezi  $x$ ,  $y$  a případně na něj  
zkusíme neuronovou sít'  
natrénovat.**

# Zdroje

- <https://medium.com/@waadlingaadil/learn-to-build-a-neural-network-from-scratch-yes-really-ca4ca457efc>
- <https://www.desmos.com/calculator>
- <https://www.sololearn.com/en/learn/courses/learn-machine-learning?location=2>
- <https://networkx.org/documentation/stable/reference/index.html>
- <https://numpy.org/doc/>
- <https://scikit-learn.org/stable/>
- <https://matplotlib.org/stable/index.html>





# Zdroje

- [https://en.wikipedia.org/wiki/Multilayer\\_perceptron](https://en.wikipedia.org/wiki/Multilayer_perceptron)
- [https://cs.wikipedia.org/wiki/Goniometrick%C3%A1\\_funkce](https://cs.wikipedia.org/wiki/Goniometrick%C3%A1_funkce)
- [https://www.researchgate.net/publication/220541319\\_Fully\\_Complex\\_Multi-Layer\\_Perceptron\\_Network\\_for\\_Nonlinear\\_Signal\\_Processing](https://www.researchgate.net/publication/220541319_Fully_Complex_Multi-Layer_Perceptron_Network_for_Nonlinear_Signal_Processing)
- <https://ai.stackexchange.com/questions/24282/what-is-the-justification-for-kaiming-he-initialization>
- <https://copilot.microsoft.com/chats/ZDZBNPYWVSiy6F8q4t917>
- <https://chatgpt.com/>
- <https://stackoverflow.com/questions/13453501/my-neural-net-learns-sin-x-but-not-cos-x>
- <https://blog.dataiku.com/pre-trained-models-ais-object-oriented-programming>



# Zdroje

- <https://news.ycombinator.com/item?id=19402666>
- <https://en.wikipedia.org/wiki/GPT-3>
- <https://seaborn.pydata.org/>
- <https://medium.com/@shauryagoel/kaiming-he-initialization-a8d9ed0b5899>
- <https://medium.com/analytics-vidhya/backpropagation-for-dummies-e069410fa585>

# Videa

- <https://www.youtube.com/watch?v=Ilq3gGewQ5U>
- <https://www.youtube.com/watch?v=WUvTyaaNkzM>
- <https://www.youtube.com/watch?v=tleHLnjs5U8>
  - ne všechny jsou o neuronových sítích



# Obrázky

- <https://www.oreilly.com/library/view/deep-learning-essentials/9781785880360/12eaae27-3de8-4f9d-8bbf-0db18d6c3297.xhtml>
- [https://sspvc.lixis.cz/matematika\\_2/mocninne\\_funkce.html](https://sspvc.lixis.cz/matematika_2/mocninne_funkce.html)
- <https://open-neuromorphic.org/neuromorphic-computing/hardware/loihi-intel/>
- <https://www.eetindia.co.in/neuromorphic-computing-system-reaches-100-million-neurons/>
- <https://aryanbajaj13.medium.com/gpt-4-is-coming-soon-heres-what-we-know-so-far-gpt-gpt2-gpt3-43c3b38eeeb8>
- <https://www.healthline.com/health/neurons>
- <https://unsplash.com/photos/blue-neurons-with-glowing-segments-over-blue-background-neuron-interface-and-computer-science-concept-3d-rendering-copy-space-G1FpRJcLoCw>
- všechny ostatní obrázky byly generovány pomocí pythónu a příslušných knihoven během tréninku sítě nebo stránkou <https://www.desmos.com/calculator>

*Děkuji za  
pozornost*







The End